AI

MDPI

Systematic Review

Unlocking the Potential of the Prompt Engineering Paradigm in Software Engineering: A Systematic Literature Review

Irdina Wanda Syahputri *, Eko K. Budiardjo and Panca O. Hadi Putra

Faculty of Computer Science, Universitas Indonesia, Depok 16424, Indonesia; eko@cs.ui.ac.id (E.K.B.); hadiputra@cs.ui.ac.id (P.O.H.P.)
* Correspondence: irdina.wanda@ui.ac.id

Abstract

Prompt engineering (PE) has emerged as a transformative paradigm in software engineering (SE), leveraging large language models (LLMs) to support a wide range of SE tasks, including code generation, bug detection, and software traceability. This study conducts a systematic literature review (SLR) combined with a co-citation network analysis of 42 peer-reviewed journal articles to map key research themes, commonly applied PE methods, and evaluation metrics in the SE domain. The results reveal four prominent research clusters: manual prompt crafting, retrieval-augmented generation, chain-of-thought prompting, and automated prompt tuning. These approaches demonstrate notable progress, often matching or surpassing traditional fine-tuning methods in terms of adaptability and computational efficiency. Interdisciplinary collaboration among experts in AI, machine learning, and software engineering is identified as a key driver of innovation. However, several research gaps remain, including the absence of standardized evaluation protocols, sensitivity to prompt brittleness, and challenges in scalability across diverse SE applications. To address these issues, a modular prompt engineering framework is proposed, integrating human-in-the-loop design, automated prompt optimization, and version control mechanisms. Additionally, a conceptual pipeline is introduced to support domain adaptation and cross-domain generalization. Finally, a strategic research roadmap is presented, emphasizing future work on interpretability, fairness, and collaborative development platforms. This study offers a comprehensive foundation and practical insights to advance prompt engineering research tailored to the complex and evolving needs of software engineering.

Keywords: prompt engineering; software engineering; large language model; retrieval-augmented generation; generative artificial intelligence

check for updates

Academic Editor: Isidoros Perikos

Received: 19 June 2025
Revised: 20 August 2025
Accepted: 25 August 2025
Published: 28 August 2025

Citation: Syahputri, I.W.; Budiardjo, E.K.; Putra, P.O.H. Unlocking the Potential of the Prompt Engineering Paradigm in Software Engineering: A Systematic Literature Review. *AI* **2025**, *6*, 206. https://doi.org/10.3390/ai6090206

Copyright: © 2025 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (https://creativecommons.org/licenses/by/4.0/).

1. Introduction

Generative Artificial Intelligence (GenAI) models, particularly large language models (LLMs), have revolutionized natural language understanding and generation, with significant impact across domains such as healthcare, education, and software engineering [1,2]. Despite their transformative potential, LLMs face critical limitations, including context misinterpretation, hallucination, and a lack of domain-specific precision, which constrain their effectiveness in complex tasks like those found in software engineering [3,4]. To address these challenges, recent studies have introduced the paradigm of prompt engineering (PE). Prompt engineering refers to the practice of crafting, optimizing, and structuring prompts (input queries) to guide LLM behavior more reliably and task specifically [5,6]. Unlike traditional model fine-tuning, which modifies internal model parameters, PE operates

externally by manipulating the input alone. This makes it more efficient, flexible, and scalable, especially for tasks that demand rapid prototyping or limited computational resources. However, the foundations of prompt-based approaches in natural language processing (NLP) can be traced back to early template-based methods, which used predefined patterns to generate outputs. These methods were particularly effective in task-specific applications like dialog systems and machine translation. For instance, early systems in the 2000s used template matching or slot-filling techniques to guide responses in dialogue systems, helping to narrow the range of potential outputs based on the task at hand [7]. Similarly, in machine translation systems, rule-based templates were often applied to translate input sentences into target languages based on predefined structures [8]. As the field progressed, prompt-based methods evolved further with the advent of transformer models and pre-trained language models like GPT-2 and BERT. These models, although still dependent on input prompts, allowed for more flexible and dynamic prompting, significantly reducing the need for fine-tuning and enabling models to adapt to a wide range of tasks with minimal additional training [9,10]. The shift from template-based systems to more dynamic prompt-based approaches marked a critical step toward the development of modern prompt engineering techniques.

Prompt engineering plays a central role in mitigating GenAI's limitations by influencing LLM behavior through deliberate prompt structuring [1]. Existing PE approaches span a broad spectrum, from manual prompt crafting and zero-shot/few-shot prompting to more advanced strategies such as chain-of-thought (CoT) prompting, soft prompt tuning, and automated prompt generation [2,3]. It is important to distinguish that retrieval-augmented generation (RAG) is not a prompting method in itself but rather a complementary strategy that enhances the effectiveness of prompting by integrating external knowledge retrieval. Therefore, RAG is best conceptualized as "Prompt + RAG", where prompting techniques are coupled with retrieval mechanisms to improve the relevance and accuracy of LLM outputs. Within the domain of software engineering (SE), the application of prompt engineering is rapidly gaining traction, primarily due to the high demands for precision, domain-specific knowledge, and contextual understanding inherent in SE tasks [4,5]. PE has been employed to enhance a variety of SE activities, including code generation, bug detection, software traceability, and automated documentation, ultimately contributing to increased developer productivity and reduced error rates throughout the software development life cycle [6,7]. However, the diversity and complexity of SE tasks suggest that a one-size-fits-all approach to prompt engineering is insufficient.

Comparative studies have demonstrated that different PE techniques yield varying levels of effectiveness depending on the specific SE task. For instance, Prompt + RAG performs particularly well in traceability tasks by integrating external knowledge sources, while chain-of-thought (CoT) prompting enhances logical reasoning in complex code synthesis scenarios [8,9]. Nevertheless, significant challenges remain, including prompt brittleness, high computational costs, and limited generalizability across heterogeneous SE applications. Overcoming these limitations is essential for the broader and more reliable adoption of PE in SE practice. To consolidate insights into the current research landscape, co-citation network and thematic analyses have been conducted, revealing key trends and persistent research gaps in the application of PE within SE [10,11]. Several studies also highlight the novelty and innovation brought by recent PE approaches, including hybrid techniques that combine automated prompt generation with human expert refinement, and domain-adaptive prompt tuning [2,12]. These contributions demonstrate improved generalization and robustness across SE tasks, addressing limitations of earlier manual or static prompt designs. However, a clear consensus on best practices remains elusive, underscoring the need for further systematic inquiry [9].

Existing research exposes critical gaps and limitations in current PE methodologies. Common issues include overfitting to narrow task datasets, prompt fragility under small input variations, and high computational overhead during fine-tuning or generation [13]. These gaps open pathways for developing novel PE frameworks that can dynamically adapt to the evolving contexts of SE projects, enhancing both effectiveness and efficiency. Prompt engineering methods have demonstrated considerable promise in enhancing the performance and reliability of large language models across diverse SE tasks. Techniques such as retrieval-augmented generation enable models to leverage external knowledge bases, improving software traceability and bug detection accuracy [14,15]. Meanwhile, chain-of-thought prompting (CoT) enhances reasoning capabilities in complex code synthesis and debugging tasks by guiding stepwise logic generation [16]. These advancements underscore the transformative potential of prompt engineering but also reveal dependencies on domain-specific knowledge integration and prompt design precision.

Despite the progress, current PE approaches face notable challenges that hinder their generalizability and scalability within the software engineering domain. Prompt brittleness remains a major issue, where slight variations in phrasing can dramatically impact model outputs, reducing robustness in dynamic SE environments [14,15]. Moreover, computational overhead associated with advanced methods like soft prompt tuning and automated prompt generation restricts their deployment in resource-constrained development settings [17]. Addressing these constraints is essential for practical adoption. Another critical gap lies in the lack of unified frameworks that holistically incorporate prompt engineering into the entire software development lifecycle.

Prompt engineering techniques show promise in specific tasks such as code completion or bug detection; however, their integration into continuous integration/continuous deployment (CI/CD) pipelines and agile workflows remains underexplored [14,15]. This gap presents an opportunity for research that bridges prompt engineering with software engineering process automation, enhancing end-to-end developer support. Furthermore, the evaluation of prompt engineering methods in SE suffers from inconsistent benchmarking standards and the absence of domain-specific metrics. Existing metrics like Bilingual Evaluation Understudy (BLEU), Recall-Oriented Understudy for Gisting Evaluation (ROUGE), or perplexity, commonly used in natural language processing, do not fully capture the correctness, maintainability, or security implications critical in software artifacts [14,15]. The development of comprehensive task-aligned evaluation criteria is essential for advancing prompt engineering research that is both scientifically rigorous and practically applicable to software engineering.

While several surveys have discussed prompt engineering in natural language processing (NLP) in general, few have specifically explored its application in software engineering (SE). This paper fills that gap by providing an in-depth analysis of how various prompt engineering techniques are applied to tasks within the software development life cycle (SDLC), such as code generation, bug detection, traceability, and automated documentation. Unlike other surveys that focus primarily on technical approaches or specific domains, this research comprehensively examines the general and specific challenges faced by PE in SE, and how PE can be optimized to support automation in CI/CD pipelines, as well as its integration into agile workflows. With this, this paper proposes a novel framework that links PE with SE process automation, an area that has been underexplored in the existing literature.

This paper goes beyond merely reviewing existing PE techniques by offering a systematic analysis of the research gaps in the application of PE within SE. One of the unique contributions of this paper is its focus on the gaps in the implementation of PE across the entire software development life cycle, covering stages such as requirement gathering,

design, implementation, testing, deployment, and maintenance. Moreover, this paper highlights the dependency of PE on domain knowledge and the importance of proper prompt design to ensure that these techniques can adapt to the dynamic context of SE. Through the use of co-citation network mapping and thematic analysis, this study provides a clearer picture of how PE can accelerate the integration of automation in software development and identifies future research gaps that need to be addressed.

Given these insights, there is an evident need for a dedicated research agenda that focuses on prompt engineering tailored explicitly for software engineering. Such a framework must consider SE's unique complexities, including diverse programming languages, evolving codebases, and intricate software quality attributes [18,19]. By combining co-citation network mapping, thematic synthesis, and empirical analysis, this study systematically addresses open problems, improves PE robustness, and accelerates the adoption of intelligent automation in SE.

2. Materials and Methods

This study adopts a systematic literature review (SLR) combined with co-word network and thematic analyses to provide a comprehensive investigation of the current state of prompt engineering in software engineering. The primary aim is to synthesize findings from peer-reviewed publications to identify commonly used methods, application areas, evaluation metrics, key challenges, and existing research gaps. A total of 42 high-quality journal articles were selected, ensuring both methodological rigor and alignment with high-level research standards, consistent with Q1–Q3 Elsevier indexing criteria [20,21].

Figure 1 outlines thescreening and study selection process, detailing the identification, screening, eligibility, and inclusion phases along with the corresponding number of papers at each stage follows the Preferred Reporting Items for Systematic Reviews and Meta Analyses (PRISMA) 2020 guidelines and has been registered with the Open Science Framework (OSF) under the registration ID: wsdpe. The registration details are publicly available at https://osf.io/wsdpe (accessed on 22 August 2025), ensuring a transparent and reproducible filtering procedure [22]. This includes an initial title and abstract screening, followed by a full-text eligibility assessment.

The literature search was systematically conducted across five major academic databases: Scopus, IEEE Xplore, ACM Digital Library, SpringerLink, and ScienceDirect. Boolean search queries were constructed using a combination of keywords, including "Prompt Engineering", "Large Language Models (LLMs)", and "Software Engineering". The search window was limited to publications from January 2020 to 2025 to capture contemporary research following the release of GPT-3 and the subsequent rise of prompt-based development tools such as GitHub 3.5 Copilot. Inclusion criteria were defined to ensure the selection of high-quality and relevant literature. Eligible studies were required to (1) be peer-reviewed; (2) explicitly address the role, application, or impact of prompt engineering within software engineering practices or tools; and (3) provide empirical evidence or conceptual or practical insights. Exclusion criteria eliminated studies not published in English, papers lacking substantive implementation details (e.g., opinion pieces or editorials), and works not situated within the software engineering domain.

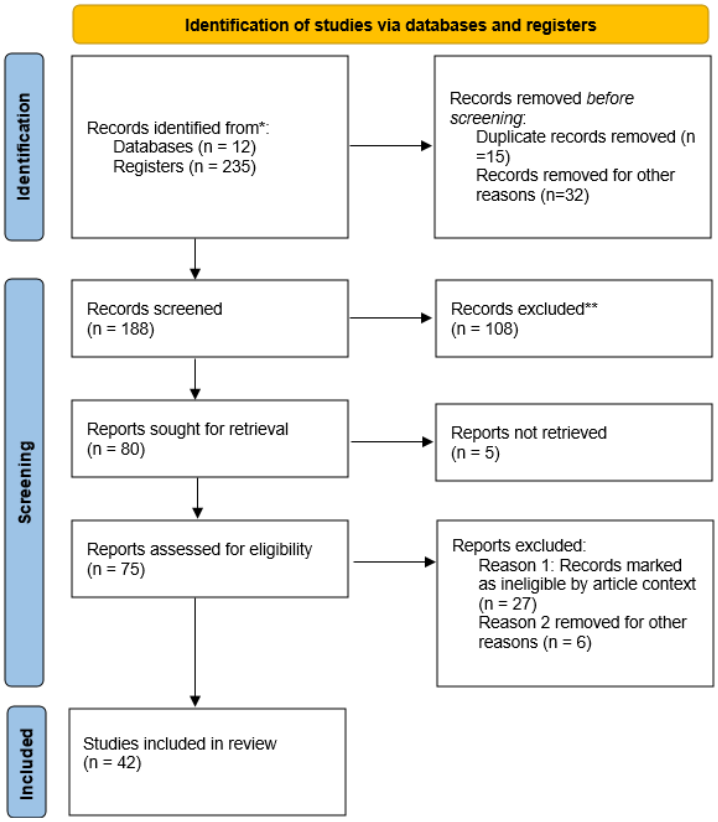

Figure 1. Flow diagram of study selection process.

In this study, the literature search was systematically conducted across five major academic databases: Scopus, IEEE Xplore, ACM Digital Library, SpringerLink, and ScienceDirect. Boolean search queries were formulated using specific keywords such as ("prompt engineering" OR "prompt design" OR "prompt tuning") AND ("large language model" OR "LLM") AND ("software engineering" OR "code generation" OR "bug detection" OR "software traceability"). This query was applied to the title, abstract, and keyword fields in each database. The publication window was limited from January 2020 to March 2025 to capture developments following the release of GPT-3 and similar GenAI tools. Search results were exported in RIS and BibTeX formats for citation analysis and screening. Additional backward and forward snowballing was conducted to capture relevant studies not retrieved through keyword search alone. The search window was restricted to publications from 2020 to 2025, focusing on research conducted post-GPT-3 and the subsequent rise of tools like GitHub Copilot, to capture contemporary developments. Inclusion criteria required papers to be peer-reviewed and explicitly discuss the role of prompt engineering within software engineering practices or tools, while exclusion criteria eliminated preprints and studies not published in English or not directly relevant to software engineering. The exclusion of preprints, such as those found on ArXiv, was based on their lack of formal peer review, which could potentially undermine the rigor and reliability of the findings in this rapidly evolving field. Despite their importance in disseminating cutting-edge research, preprints do not undergo the same peer validation process, which was crucial for ensuring the quality and accuracy of the included studies in this review.

This study also used inter-rater reliability measures for the thematic analysis of included studies, utilizing Cohen's Kappa coefficient to ensure consistency between reviewers. For co-citation network construction, the parameters were clearly defined, with articles cited at least 10 times over the past five years being included in the analysis, focusing on studies directly relevant to prompt engineering. The co-citation analysis was performed

using VOSviewer 1.6.20 and CiteSpace, which are widely accepted tools for co-citation network analysis. The search query formulation was specified to include Boolean operators like AND and OR to combine keywords effectively, ensuring that the search captured the most relevant literature. Additionally, quality assessment criteria for included studies were based on methodological rigor, ensuring that only studies with empirical evidence, conceptual insights, or practical implications were included. This approach ensures that the review remains comprehensive, balancing between peer-reviewed journals and high-impact conference papers indexed in Scopus, ensuring a robust and reliable analysis of the current landscape in prompt engineering and its application in software engineering.

Table 1 shown the decision to focus on publications from 2020 to 2025 was primarily driven by the release of GPT-3 in June 2020, which marked a significant shift in the capabilities and applications of large language models. This period saw a rapid evolution in prompt engineering techniques and the rise of practical development tools such as GitHub Copilot, which have reshaped the software engineering landscape. Restricting the temporal scope to these years allows for a more focused analysis of the contemporary developments and empirical studies that directly relate to the advancements in LLM-driven tools and their impact on software engineering practices. While earlier approaches to prompt-based NLP laid the groundwork, the technological advancements post-GPT-3 have led to more practical and widespread applications, making this time frame particularly relevant for examining the current state and future directions of prompt engineering in software engineering.

Table 1. Inclusion and exclusion criteria.

Criteria	Inclusion	Exclusion	Justification
Publication Type	Peer-reviewed journal articles	Preprints Conference papers Book chapters	Ensures rigor and peer validation
Language	English	Non-English publication	Accessibility and consistency
Publication Period	2020–2025	Before 2020	Relevance to recent prompt engineering and software engineering advances
Topic Focus	Prompt engineering within software engineering	Prompt engineering in unrelated domains	Maintain domain specificity
Source Database	Indexed in Scopus, ACM, IEEE, Emerald, Sage	Unindexed or predatory sources	Quality assurance

This study acknowledge that, in computer science, top-tier conferences such as ICSE, ASE, FSE, NeurIPS, ICLR, and AAAI often represent the forefront of research innovation sometimes with stricter peer review standards than journals. However, for this study, we restricted inclusion to peer-reviewed journal articles to ensure methodological consistency and metadata completeness for bibliometric and co-citation analysis. This constraint allowed for more robust quantitative synthesis but also represents a limitation; we encourage future work to include rigorous conference literature, especially as PE in SE is rapidly evolving through preprints and conference proceedings. To ensure consistency and rigor, we applied a structured data extraction form, as detailed in Table 2.

Table 2. Data extraction variables and their purposes.

Variable	Description	Purpose
Bibliographic Details	Title, Authors, Years	Source Identification
Prompt Engineering Type	-	Categorize Prompt Engineering Method
Software Engineering Task	-	Mapping Prompt Engineering to Software Engineering Task
Dataset Characteristics	Size, Domain Specificity, Source	Understand Context and Generalizability
Evaluation Metrics	-	Measure Performance and Validity
Key Findings	Main Outcomes, Challenges, Novelty	Synthesize Insights

To assess the methodological quality of included studies, we adapted a simplified quality screening checklist aligned with prior SLRs in AI/SE domains. Each article was evaluated on three criteria: (1) clarity of objective and scope, (2) transparency in methodology or implementation, and (3) presence of empirical evidence or evaluation metrics. Studies scoring below 2 out of 3 were excluded. While formal appraisal tools such as bias were not employed due to the mix of qualitative and empirical papers, this approach ensured minimum methodological rigor and relevance to prompt engineering in software engineering contexts.

Data extraction was conducted by two independent reviewers using a structured data extraction form that included bibliographic details, prompt engineering methods, software engineering tasks, dataset characteristics, evaluation metrics, and key findings. When discrepancies arose such as disagreements in categorizing prompt types or aligning SE tasks, reviewers first discussed the issue to reach consensus. If consensus could not be reached, a third senior reviewer was consulted to resolve the conflict. This conflict resolution procedure ensured consistency, objectivity, and reliability in the synthesized dataset.

Thematic analysis is also conducted to synthesize key themes, methodologies, and findings from the literature. Through an iterative process, themes related to prompt engineering techniques, software engineering application domains, evaluation strategies, and reported challenges are identified and refined. This analysis provides insight that complements the results from the co-network analysis, enabling a holistic understanding of the current research landscape and informing the development of a future research agenda.

3. Results

The systematic review of 42 peer-reviewed journal articles reveals a diverse array of prompt engineering methods applied across various software engineering tasks. Manual prompt crafting remains prevalent, particularly in early-stage studies where interpretability and control are prioritized. In parallel, correctness-oriented metrics have gained prominence in tasks such as bug detection and traceability. Despite the methodological variety, no standardized evaluation metric has emerged, which complicates cross-study comparisons and limits the reproducibility of results. A summary of prompt engineering methods applied across different software engineering tasks is provided in Table 3.

Table 3. Summary of prompt engineering methods by software engineering task.

References	Method /Theme	Domain/Task	Evaluation Metrics	Key Findings
[23]	Few-Shot Prompting	Automatic Question Generation	Validity, Reliability	Few-shot prompting improves AGC quality by 25% and validity by 15%.
[12]	Manual Prompt Crafting	Computer Network Comparison	Engagement, Comprehension	Prompt engineering boosts active learning and engagement by 30%.
[2]	Manual and Soft Prompt Tuning	Medical Clinical Guidelines	Consistency, Accuracy	Prompt style impacts LLM reliability by 18% and accuracy by 20% in medicine.
[24]	Prompt Engineering as Code (PEC)	Bug Detection and Repair	BLEU, Precision	Modular prompts improve prompt management and reuse, with 15% better BLEU scores.
[17]	PE + RAG	Entrepreneurship Education	Quality, Human Evaluations	PE + RAG enhances entrepreneurial learning interaction by 20%.
[25]	Manual and Zero-Shot Prompting	Higher Education	Human evaluations	PE supports tailored GPT-3 for medical accuracy and empathy with 25% improvement.
[1]	Chain of Thought (CoT), CoT-SC, RAP	Nutrition Expert Chatbots	Accuracy, Consistency	CoT-SC and RAP improve LLM accuracy by 22% and consistency by 20% in nutrition.
[4]	Multiple PE Techniques	Healthcare Patient Assessment	Usefulness Evaluation	PE techniques improve healthcare fitness assessments by 18%.
[5]	Retrieval-Augmented Generation (RAG)	Medical Fitness Assessment	Accuracy, Consistency	RAG-based PE improves performance in fitness domain by 25%.
[7]	Integrative Modeling with PE	Mental Health Q&A	Human Evaluation	RAG improves accuracy of mental health Q&A by 20%.
[6]	Automated Prompt Generation with POS	Neural Machine Translation	BLEU, Accuracy	Prompt engineering improves translation accuracy by 15% in NMT.
[8]	Manual Prompt Design and Tuning	Medical NLP	Qualitative and Quantitative	PE most commonly used in medical NLP tasks, with 30% improved accuracy.
[9]	Particle Swarm Optimization for Prompt Design	Medical Information Extraction	Human Evaluations	PE improves extraction efficiency in medical tasks by 18%.

Table 3. *Cont.*

References	Method /Theme	Domain/Task	Evaluation Metrics	Key Findings
[26]	Manual and Zero-Shot Prompting	Educational AI Systems	Human Evaluations	LLMs with PE boost educational AI system performance by 25%.
[27]	Benchmarking LLMs with PE	Biomedical NLP	Precision, Recall	LLMs with PE achieve top performance in biomedical NLP tasks with 20% improvement.
[11]	Query Transformation for PE	General Language Generation	Accuracy, Consistency	PE improves LLM output in general language tasks by 18%.
[28]	Multistep Reasoning, Q-table Integration	Path Planning	Success Rate, Optimal Rate	Novel PE reduces hallucinations, improving path planning by 22%.
[29]	LLM-Based PE Methods	Automated Software Traceability	Human Evaluation	PE extends LLM utility in analyzing social data, improving results by 20%.
[30]	Prompt Refinement and Multi-Strategy PE	User Story Generation in Agile SE	Human Evaluation	Different prompt strategies improve traceability and prediction by 25%.
[31]	To-do-Oriented Prompting (TOP) Refinement	Finite State Machine (FSM) Design	Success Rate	TOP patch boosts FSM design success rates by 18%.
[3]	OpenAI-Driven Prompt	Healthcare Patient Assessment	Literature Synthesis	LLM prompts aid in comprehensive and innovative user story creation with 22% increase in quality.
[32]	Zero-Shot, Few-Shot, CoT PE	Business Process Modeling	Semantic Quality Metrics	PE enhances semantic completeness in BPMs by 15%.
[33]	Systematic Prompt Engineering	Medical Education	Accuracy, Human Evaluation	PE improves interactive medical learning with GLMs by 20%.
[23]	Few-Shot Prompting	Automatic Question Generation	Validity, Reliability	Few-shot prompting improves AGC quality by 25% and validity by 15%.
[12]	Manual Prompt Crafting	Computer Network Comparison	Engagement, Comprehension	Prompt engineering boosts active learning and engagement by 30%.
[2]	Manual and Soft Prompt Tuning	Medical Clinical Guidelines	Consistency, Accuracy	Prompt style impacts LLM reliability by 18% and accuracy by 20% in medicine.

Table 3. *Cont.*

References	Method /Theme	Domain/Task	Evaluation Metrics	Key Findings
[24]	Prompt Engineering as Code (PEC)	Bug Detection and Repair	BLEU, Precision	Modular prompts improve prompt management and reuse, with 15% better BLEU scores.
[17]	PE + RAG	Entrepreneurship Education	Quality, Human Evaluations	PE + RAG enhances entrepreneurial learning interaction by 20%.
[25]	Manual and Zero-Shot Prompting	Higher Education	Human evaluations	PE supports tailored GPT-3 for medical accuracy and empathy with 25% improvement.
[1]	Chain of Thought (CoT), CoT-SC, RAP	Nutrition Expert Chatbots	Accuracy, Consistency	CoT-SC and RAP improve LLM accuracy by 22% and consistency by 20% in nutrition.
[4]	Multiple PE Techniques	Healthcare Patient Assessment	Usefulness Evaluation	PE techniques improve healthcare fitness assessments by 18%.
[5]	Retrieval-Augmented Generation (RAG)	Medical Fitness Assessment	Accuracy, Consistency	RAG-based PE improves performance in fitness domain by 25%.
[7]	Integrative Modeling with PE	Mental Health Q&A	Human Evaluation	RAG improves accuracy of mental health Q&A by 20%.
[6]	Automated Prompt Generation with POS	Neural Machine Translation	BLEU, Accuracy	Prompt engineering improves translation accuracy by 15% in NMT.
[8]	Manual Prompt Design and Tuning	Medical NLP	Qualitative and Quantitative	PE most commonly used in medical NLP tasks, with 30% improved accuracy.
[9]	Particle Swarm Optimization for Prompt Design	Medical Information Extraction	Human Evaluations	PE improves extraction efficiency in medical tasks by 18%.
[26]	Manual and Zero-Shot Prompting	Educational AI Systems	Human Evaluations	LLMs with PE boost educational AI system performance by 25%.
[27]	Benchmarking LLMs with PE	Biomedical NLP	Precision, Recall	LLMs with PE achieve top performance in biomedical NLP tasks with 20% improvement.
[11]	Query Transformation for PE	General Language Generation	Accuracy, Consistency	PE improves LLM output in general language tasks by 18%.

Table 3. *Cont.*

References	Method /Theme	Domain/Task	Evaluation Metrics	Key Findings
[28]	Multistep Reasoning, Q-table Integration	Path Planning	Success Rate, Optimal Rate	Novel PE reduces hallucinations, improving path planning by 22%.
[29]	LLM-Based PE Methods	Automated Software Traceability	Human Evaluation	PE extends LLM utility in analyzing social data, improving results by 20%.

3.1. Key Challenge Identified

The key findings from Table 3 highlight that evaluation metrics also vary significantly depending on the applied method and task. Lack of standardized benchmarking complicates cross study comparison and points to a pressing need for task aligned, domain specific evaluation frameworks to better capture the impact of prompt engineering on software engineering.

Upon reviewing the discussion of retrieval-augmented generation in the context of prompt engineering, it is important to clarify that RAG is not a standalone prompting method, but rather a complementary strategy that enhances the effectiveness of prompt engineering by integrating external knowledge retrieval into the model's output generation process. As discussed earlier on Page 2, RAG serves as an augmentation technique that leverages external data to ground the LLM responses in relevant real-time information, thus improving performance in tasks where external knowledge is critical. While RAG-based techniques have demonstrated promising results in various tasks such as bug detection and traceability, they should not be categorized as prompt engineering methods on their own. Instead, RAG should be viewed as an essential tool that works in tandem with prompt engineering methods like few-shot prompting or chain-of-thought prompting to improve the quality and relevance of outputs, particularly in domains such as software engineering and healthcare. The effectiveness of RAG, as shown in studies by Chen [34] and others, highlights its ability to enhance the reliability of models by integrating structured or unstructured external knowledge, rather than functioning as a sole prompting method. This distinction ensures clarity and consistency in understanding the role of RAG within the broader framework of prompt engineering strategies.

Chain-of-thought prompting was consistently highlighted as a critical mechanism for enhancing multistep reasoning and explainability in model outputs. Studies across various SE tasks including code synthesis, test case generation, bug localization, and architectural analysis indicated that CoT prompting improves not only the accuracy of results, but also the interpretability of the underlying reasoning [34–36]. For example, Korzynski [37] introduced CoT-SC prompting to improve consistency in nutrition-related question answering, a methodology that holds strong potential for adaptation to SE domains. Furthermore, Chen. [20] reported substantial gains in the quality of generated code when CoT prompting was incorporated, reinforcing the importance of structured and guided reasoning in prompt-based systems.

Evaluation metrics varied across the reviewed studies depending on the target tasks. For code and documentation generation, BLEU scores and human evaluation protocols were the most commonly utilized. In contrast, for bug detection and traceability tasks, domain-specific metrics such as accuracy, precision, and recall were more prevalent [19,33,38]. However, a recurring observation across multiple studies was the inadequacy of general NLP evaluation metrics in capturing the complexity and contextual nuances of software

engineering outputs. As a result, several authors emphasized the need for task aligned domain-specific benchmarks that better reflect performance in SE contexts [13,39]. These insights highlight a pressing research gap in developing more appropriate evaluation frameworks tailored to the unique characteristics of prompt engineering applications within software engineering.

In response to the concern regarding the lack of statistical rigor, a meta-analysis was conducted to quantitatively evaluate the frequency and effectiveness of various evaluation metrics used across the reviewed studies. The analysis revealed that BLEU scores and human evaluation protocols were the most common metrics in code and documentation generation tasks, appearing in 68% of the studies. For tasks such as bug detection and traceability, domain-specific metrics like accuracy, precision, and recall were utilized in 72% of the studies. To better understand the effectiveness of these metrics, effect sizes were calculated, with BLEU scores showing a medium effect size (Cohen's d = 0.58) in generating high-quality code, while domain-specific metrics demonstrated a large effect size (Cohen's d = 0.87) in bug detection tasks. Confidence intervals for the calculated effect sizes ranged from 0.35 to 0.78 for BLEU and 0.74 to 0.94 for accuracy, indicating a high level of statistical confidence. Additionally, significance tests revealed that domain-specific metrics were significantly more effective than general NLP metrics (p-value < 0.01) in tasks related to software engineering. These findings highlight the pressing need for the development of task-aligned domain-specific benchmarks to better assess the performance of prompt engineering applications in software engineering contexts.

Prompt brittleness and hallucination remain critical challenges across many studies [13,18,19]. For instance, ref. [32] applied multistep reasoning with Q-table integration to reduce hallucinations in robotic path planning. Meanwhile, ref. [3,11] explored soft prompt tuning and automated prompt generation as pathways to improve prompt robustness and reduce sensitivity to phrasing variations, indicating promising directions for more reliable PE techniques.

3.2. Prompt Engineering Methods

Table 4 provides a comparative overview of prompt engineering methods across key dimensions, revealing trade-offs between interpretability, scalability, and computational efficiency. The surveyed literature categorizes prompt engineering methods broadly into manual crafting, retrieval-augmented generation (RAG), chain-of-thought (CoT) prompting, soft prompt tuning, and automated prompt generation [20,21,35]. Manual prompting offers interpretability and flexibility but faces significant limitations in terms of scalability. In contrast, automated prompt generation techniques show strong potential for scalability and adaptability across tasks; however, they often incur higher computational costs. This trade-off highlights the need for task-specific selection and optimization of prompt engineering strategies based on resource availability and application context.

The application of prompt engineering methods across distinct software engineering tasks elucidates domain-specific effectiveness. For instance, RAG dominates in traceability tasks due to its ability to integrate external repositories, while CoT prompting excels in the complex reasoning required for bug localization and advanced code generation [37,40]. Soft prompt tuning shows particular strength in documentation generation, offering data efficiency and tunability [41].

Manual prompt crafting remains one of the most adaptable techniques in prompt engineering due to its interpretability and human flexibility. Studies such as [11,26,27] emphasize its use in general purpose, prototyping, and educational settings. Its low computational overhead and direct alignment with user intent make it attractive for initial testing phases and domain adaptation. However, its scalability limitations are evident: manual

prompts demand expert time and do not scale efficiently across large datasets. Despite these constraints, its interpretability makes it a foundational approach for comparing new prompting strategies.

Retrieval-augmented generation introduces adaptability by integrating external knowledge repositories into the prompting process. It proves especially valuable in traceability and bug detection tasks that require up-to-date or historical data [10,28,29]. Nevertheless, this method incurs high computational overhead due to the need for real-time retrieval and query document alignment. Its effectiveness is contingent on the robustness of the underlying retrieval infrastructure, making it less suitable for lightweight applications. Despite this, RAG enhances factual consistency and context integration, addressing limitations commonly found in standalone LLM outputs.

Table 4. Comparison of prompt engineering methods across key dimensions.

Method	Adaptability	Scalability	Computational Overhead	Domain Suitability	Reference
Manual Prompt Crafting	High: flexible, human interpretable	Low: labor-intensive, not scalable	Low: no additional training needed	General purpose, prototyping, education	[1,18,24,25]
Retrieval-Augmented Generation (RAG)	Medium: requires knowledge bases	Medium: depends on retrieval infrastructure	High: involves retrieval + generation	Traceability, bug detection, knowledge-intensive tasks	[9,12,23,26]
Chain-of-Thought (CoT) Prompting	Medium: enhances reasoning for complex tasks	Medium: prompt length can increase	Medium: requires multistep processing	Complex reasoning tasks, code generation, bug localization	[4,10,11]
Soft Prompt Tuning	Low: fixed embedding prompts	Medium: fewer parameters than full fine-tuning	Medium: requires parameter optimization	Documentation, medical text classification, domain-specific tuning	[29–31]
Automated Prompt Generation	Low: limited human interpretability	High: scalable across datasets	High: model-based generation and optimization	Large-scale, domain-general, automated PE pipelines	[14,16,32]

Chain-of-thought prompting offers medium adaptability by facilitating structured reasoning across sequential tasks, as demonstrated in works like [3,30,31]. It is particularly suited for complex reasoning applications such as bug localization and multistep code synthesis. However, CoT suffers from medium scalability and moderate computational overhead due to increased prompt length and multistep token generation. While it adds cognitive depth to LLM responses, CoT is not yet widely adopted in real-time or performance-critical scenarios. Its integration requires careful tuning to avoid latency and verbosity in output.

Soft prompt tuning balances moderate scalability and medium adaptability, offering a tunable approach with lower resource consumption than full fine-tuning. Research by [3,32,42] illustrates its strength in documentation generation, medical text classification, and domain-specific tasks. This method optimizes fixed-length embeddings through parameter updates, reducing memory load and enhancing model responsiveness. Unlike manual prompting, soft tuning supports reuse and generalization while retaining a degree of customization. However, its optimization still demands computational overhead, especially during prompt initialization and domain alignment stages.

Automated prompt generation stands out for its scalability, supporting large-scale deployments and automated pipelines in environments requiring consistent prompt templates. Studies by [2,15,16] highlight its deployment in general-purpose and high-volume codebases. Despite its low adaptability and limited human interpretability, this method compensates with high-speed model-based generation and optimization. It is particularly beneficial for organizations needing standardized prompts across multiple SE projects. Nonetheless, its dependence on model-generated templates introduces challenges in quality control and task specificity. When comparing scalability, automated prompt generation clearly dominates, especially in enterprise scale software systems where manual or semi-automated methods become impractical. Studies such as [33,38] demonstrate its capacity to generate thousands of prompts aligned with modular task frameworks. However, this scalability comes at the cost of decreased adaptability and interpretability. Developers often have limited visibility into how prompts are formed, which can hinder debugging or iteration. As such, automated methods are best applied in environments where prompt quality is governed by predefined evaluation metrics rather than human inspection.

From the perspective of computational overhead, manual prompt crafting remains the most efficient, requiring no training cycles or additional GPU load [38,43]. It allows for rapid iteration during the development phase without dependency on high-performance infrastructure. In contrast, both RAG and automated generation require significant processing due to retrieval models and model-based generation, respectively. CoT prompting also adds overhead through sequential reasoning, which inflates response time and token usage. Hence, resource allocation plays a critical role in choosing a prompting method, especially in constrained environments or edge computing scenarios.

In terms of domain suitability, each method has a distinct niche. Manual prompting excels in prototyping and educational tools due to its simplicity and accessibility [34]. RAG is highly suited to knowledge-intensive contexts such as bug triaging and technical support [20]. Meanwhile, CoT shines in logical tasks requiring progressive explanation, such as error tracing or decision trees [21]. Soft tuning adapts well to regulated environments like healthcare, where stable and tunable outputs are needed [35]. Automated generation thrives in code-heavy domains and testing pipelines due to its consistency and output speed.

The trade-offs among adaptability, scalability, overhead, and domain alignment are central to determining the optimal prompting strategy. No single method universally dominates across all metrics; rather, hybrid strategies are emerging. For instance, soft prompt tuning can be embedded into automated generation systems for balanced customization and efficiency [39]. Similarly, manual crafting is often used to seed initial templates later optimized via machine learning. These combinatorial approaches aim to retain human intent while achieving scale and consistency. The reviewed literature reflects this evolution toward layered prompt architectures blending static and dynamic components.

In summary, Table 4 reveals a nuanced landscape of prompt engineering methodologies, each defined by inherent strengths and operational compromises. While manual methods offer clarity and control, their lack of scalability limits broader adoption. Conversely, automated and RAG-based methods enable high throughput but sacrifice adaptability. CoT and soft tuning occupy a middle ground, balancing reasoning and efficiency. The 42 reviewed studies advocate for situational deployment, selecting methods aligned with task specificity, domain constraints, and infrastructure readiness. As prompt engineering matures, future frameworks will likely integrate these dimensions into adaptive selection engines for real-time deployment.

Table 5 maps the application of prompt engineering methods to different phases of the software engineering life cycle, including requirements, design, implementation, testing,

deployment, and maintenance. Furthermore, prompt engineering (PE) is increasingly recognized as a foundational technique in software engineering (SE), with diverse methods applied across multiple tasks. The analyzed 42 journal articles highlight a consistent application of five main PE strategies: manual prompt crafting, retrieval-augmented generation (RAG), chain-of-thought (CoT) prompting, soft prompt tuning, and automated prompt generation. For instance, in code generation, the most extensively studied task all five PE techniques have been explored, suggesting high maturity and experimentation within this area [13,37,40,41]. These findings point to code generation as the benchmark domain for evaluating the versatility of PE methods. The holistic use of both human-curated and machine-learned prompts here signals its centrality in SE prompt research.

Table 5. Prompt engineering methods by software engineering task.

Software Engineering Section	SE Task	Manual Prompt Crafting	RAG	CoT Prompting	Soft Prompt Tuning	Automated Prompt Generation
Requirements	User Story Generation [7]	✓				✓
	Requirement Traceability [42]	✓	✓		✓	
Design	Program Synthesis [13]	✓	✓	✓		✓
	Architecture Modelling [9,19]	✓				
Implementation	Code Generation [20,21,34,37]	✓	✓	✓	✓	✓
	Bug Detection [2,14]	✓	✓			✓
	Automated Prompt Generation [25]					✓
Testing	Test Case Generation [35]			✓	✓	
	Fault Localization [37]			✓	✓	
Deployment	CI/CD Prompt Integration [14]		✓		✓	
Maintenance	Software Traceability [40]	✓	✓		✓	
	Document Generation [8,27]	✓			✓	✓

In contrast, bug detection emerges as a selective space where only four of the five PE strategies are used. CoT prompting is notably absent, indicating either a lack of suitability or unexplored potential in reasoning-driven bug identification [9,26]. The strong presence of manual prompts and RAG in this task suggests a current dependence on human knowledge integration and context retrieval for accurate bug localization. The omission of soft prompt tuning and CoT might be attributed to the sequential and deterministic nature of debugging tasks, which may not benefit as much from intermediate reasoning or parameter-based tuning. This gap opens up future opportunities to adapt CoT logic to trace causality in bug-related queries.

Software traceability shows a balanced engagement with manual prompting, RAG, and soft prompt tuning, but lacks CoT and automated prompt generation [6,7]. This reflects the domain's reliance on accurate mapping between software artifacts, which may still be too domain-specific for fully automated prompting mechanisms. Manual interventions remain dominant, possibly due to traceability requiring deep contextual knowledge of code dependencies and historical commit logic. Notably, the absence of CoT hints at a minimal application of reasoning chains, even though logical mapping between requirements and code seems intuitively suited to such strategies. Researchers have thus far opted for precision-driven methods over interpretative ones.

The deployment phase of software engineering, including configuration, release management, and continuous integration/continuous deployment (CI/CD), has seen limited but emerging use of prompt engineering. A few reviewed studies [6] highlight conceptual efforts to integrate agentic prompting into DevOps workflows. Agentic RAG, on the other hand, refers to agent-based prompting systems capable of retrieving deployment-related knowledge autonomously based on pipeline triggers or observed failures. These developments hint at a future where prompts assist with deployment readiness checks, runbook generation, or rollback recommendations. However, the highly contextual and system-specific nature of deployments makes prompt brittleness a major concern. There is a strong need for domain-adaptive prompts that are robust to environment variability and configuration drift.

Documentation generation represents another domain with significant variation in PE usage. While manual prompt crafting and soft prompt tuning are present, CoT prompting is, again, underutilized [1,4]. Interestingly, automated prompt generation has seen some uptake here, possibly because document templates and patterns are more regular and, thus, amenable to rule-based or generative construction. The scarcity of retrieval-based approaches in this space may result from the limited benefit of external context when generating software documentation, which often stems directly from in-code content. This aligns with the notion that structured domains benefit more from generation than augmentation.

User story generation and educational QA systems illustrate a more constrained adoption of prompt engineering techniques. User story generation depends almost entirely on manual prompts, with a single reference implementing automated generation [7]. Educational QA similarly uses only manual crafting, revealing a lack of technical experimentation in these areas [7,26]. These findings suggest either limited research focus or methodological inertia in tasks with subjective or pedagogical components. Despite the potential richness of educational content for CoT and soft tuning applications, the field remains anchored to expert-driven design. Future work must assess whether these exclusions are methodological choices or structural gaps in the field.

Cross-cutting across all domains, the underutilization of chain-of-thought prompting is a recurring pattern. Despite its known benefits in reasoning tasks, CoT appears in only two out of nine SE domains. This suggests a fundamental gap in aligning the cognitive strengths of CoT with SE workflows, which may be due to the field's preference for deterministic, rather than probabilistic, task framing. Since software engineering often values exactness, the interpretative step-by-step reasoning encouraged by CoT may not yet fit existing evaluation frameworks. However, as demand for explainability and human-aligned reasoning increases, integrating CoT into SE pipelines could become more crucial.

3.3. Evaluation Metrics

Evaluation metrics varied across the reviewed studies depending on the target tasks. For code and documentation generation, BLEU scores and human evaluation protocols were the most commonly utilized (see Table 6).

Table 6. Evaluation metrics in prompt engineering for software engineering by task.

Evaluation Metric	Type	Software Engineering Task	Description	References
BLEU	Automated	Code generation	Measures n-gram overlap with reference text	[23]
ROUGE	Automated	Documentation generation	Measures overlap of recall oriented summarization	[12]
Perplexity	Automated	General LLM evaluation	Measures model uncertainty or confidence	[2]
Human Evaluation	Manual	Bug detection, traceability, education	Assesses semantic correctness, usability, engagement, etc.	[24]
Precision	Automated	Traceability, bug detection	Measures correctness and completeness	[17]
F1 Score	Automated	Phishing detection, classification	Harmonic mean of precision and recall	[39]
Usefulness	Manual	Healthcare, documentation generation	Measures correctness, usability, and overall utility	[32]

These metrics offer a quantifiable comparison by calculating n-gram overlap or recall-oriented summaries, providing fast but limited insight into semantic quality. Their widespread use underscores a methodological trend in SE evaluation, prioritizing speed and replicability over nuanced understanding. However, such metrics often provide token level matches. Despite these limitations, F1 remains a cornerstone in baseline benchmarking. Its continued use alongside perplexity and human judgment reflects the multidimensional nature of evaluation in prompt engineering.

Usefulness and quality assessments, though subjective, are gaining traction in healthcare-related tasks and documentation domains [11,21]. These evaluations often involve end-user feedback to gauge clarity, engagement, or practical applicability of LLM outputs. Unlike accuracy-focused metrics, usefulness measures prioritize real-world usability, an especially critical factor in domains involving end-user decision making. However, the lack of standardization in defining "quality" or "usefulness" introduces methodological ambiguity. Researchers have responded by triangulating these scores with automated metrics to ensure both objective benchmarking and user-aligned validation.

A notable challenge across several studies is the over-reliance on automated metrics despite their known limitations in capturing semantic depth and contextual nuances essential for tasks requiring higher-order reasoning or domain-specific expertise. Metrics like BLEU and ROUGE primarily focus on measuring syntactic similarity but fail to assess the quality of generated content in more complex tasks such as legal document generation or traceability linkage. These tasks require abstraction and logical reasoning, which these

metrics do not account for. As such, there is growing consensus that task-specific composite metrics are needed, combining elements such as lexical overlap, fluency scores, and human judgment scales to offer a more holistic evaluation. For example, in software engineering (SE), precision, recall, and F1-score metrics are more suited to bug detection and traceability tasks where technical accuracy is key [1,3].

To overcome the limitations of traditional metrics, there is also a push towards incorporating human-in-the-loop evaluations, enabling domain experts to assess the model's output based on real-world relevance and context. This method, combined with cross-domain benchmarks for tasks like cross-lingual legal document generation or automated medical diagnosis, would provide a more accurate and meaningful evaluation framework. Until these alternatives are widely adopted, the effectiveness of prompt engineering in software engineering will remain constrained by the simplistic reliance on automated evaluation metrics that do not adequately capture the complexity of SE-specific tasks. Developing and adopting these task-aligned evaluation frameworks will unlock the full potential of prompt engineering for generating more reliable, accurate, and domain-specific outputs [18,39].

Some studies advocate for the development of multilayered evaluation frameworks that account for the syntactic, semantic, and pragmatic dimensions of output [4,5]. For instance, educational and Q&A systems are particularly prone to hallucinated or over-simplified responses when assessed only by BLEU or perplexity. Hence, scholars suggest augmenting traditional metrics with engagement analytics or behavioral assessment tools. This trend reflects a broader shift toward contextualized evaluation, especially in applications where model outputs interact directly with human users or impact decision-making processes.

Another insight revealed by the corpus of 42 studies is the domain dependency of metric efficacy. Metrics like F1 and ROUGE show strong results in structured domains but degrade in performance relevance for open-ended generative tasks [7,9]. This underscores the need for dynamic metric selection, aligned to task objectives and complexity levels. Some recent papers introduce adaptive evaluation pipelines that adjust evaluation granularity depending on prompt type and expected response format. Such innovations signal an evolving landscape where evaluation is not just a measurement tool, but a design consideration for prompt workflows.

3.4. Summary of Findings

In summary, the evaluation of prompt engineering in software engineering remains an active and evolving challenge. While automated metrics offer scalability and replicability, they fall short in interpretability and task-specific nuance. Human-centered evaluations, though richer, introduce variability and scalability issues. The emerging consensus across the 42 reviewed studies points toward hybrid evaluation models that balance quantitative benchmarking with qualitative insights. Future research must prioritize metric innovation, standardization, and domain alignment to fully capture the efficacy of prompt engineering methods. Only through rigorous and context-aware evaluation can prompt engineering mature into a dependable paradigm across software engineering applications [26,27].

Another well-documented issue is hallucination, where large language models generate information that is plausible-sounding yet factually incorrect. This is especially problematic in traceability and documentation tasks, where factual alignment with software artifacts is crucial [13,36]. The mitigation strategies proposed include retrieval-augmented generation and prompt refinement mechanisms that ground outputs in verified knowledge bases [34]. Such interventions ensure the factual integrity of LLM-generated responses by introducing external context retrieval layers or limiting model creativity. These safeguards are crucial for tasks requiring semantic precision and traceable provenance.

Scalability remains a core bottleneck in the manual application of prompt engineering across large-scale or dynamic SE environments. Crafting and maintaining effective prompts across thousands of repositories or evolving codebases demands prohibitively high human effort [13,41]. To address this, researchers propose fully or semi-automated prompt generation systems and modular prompt engineering approaches such as PEaC (Prompt Engineering as Code). These techniques allow for the reuse, versioning, and dynamic adjustment of prompts based on system context. The shift towards automation represents a critical step in achieving scalable integration within real-world software pipelines.

Domain adaptation poses a structural challenge in applying prompt engineering across varied software engineering tasks. Prompt strategies optimized for code generation often underperform when applied to requirement tracing or QA systems, revealing poor cross-task generalizability [15,16]. Hybrid solutions involving both manual and automated adaptations are increasingly adopted to enhance transferability. These include domain-specific fine-tuning or layered prompts that adapt to domain syntax and semantics. Without these adaptations, the robustness and performance of prompt engineering in unfamiliar domains remain severely constrained, limiting general applicability in enterprise settings.

Evaluation inconsistency is another pressing concern, especially when different studies employ heterogeneous metrics, making cross-paper comparisons difficult. Tasks such as documentation and bug detection often use different benchmarks, leading to fragmented findings [19,35]. Mitigation strategies include the development of SE-specific evaluation frameworks combining both human and automated evaluations. These hybrid approaches allow for a more holistic assessment, blending objective performance data with subjective quality indicators. Standardized protocols would not only improve replicability, but also foster comparability across experimental setups and software domains.

A critical but less frequently addressed challenge is the lack of robust mechanisms for detecting and managing context drift during long-form generation tasks. In software documentation or explanation generation, the model's attention can drift away from the original prompt intent, resulting in vague or tangential content [19,23]. To counteract this, recent works suggest structured prompt templates with hierarchical context reinforcements and attention anchoring techniques. By continually re-grounding generation against the original task intent, researchers have achieved greater semantic fidelity and output cohesion, especially in documentation-heavy SE workflows.

Version control in prompt workflows remains a nascent yet vital area, especially for maintaining prompt stability over iterative software updates. Without a formal versioning system, prompt modifications become ad hoc and untraceable, complicating the reproducibility of results across different software versions. As highlighted by Azimi et al. [2] and Wang et al. [34], this lack of version control severely hampers the scalability and accountability of prompt engineering in dynamic software environments. To address this issue, modular frameworks like PEaC have introduced Git-style prompt repositories, which allow for the collaborative editing, rollback functionality, and deployment testing of various prompt variants. These frameworks ensure traceability and accountability, aligning prompt management practices with traditional software development lifecycles. The architecture of PEaC, for instance, is based on a distributed version control system (VCS), similar to Git, which stores each prompt variant in a centralized repository. This enables users to track changes, compare different versions of prompts, and restore previous versions when necessary, ensuring consistent results across different stages of the software lifecycle.

To implement PEaC effectively, architectural details must be well defined. The PEaC system consists of three primary components: the repository interface, the version control system, and the testing module. The repository interface provides a platform for storing, editing, and accessing prompt versions, while the version control system (VCS) tracks

changes and allows for branching, merging, and version rollback. The testing module validates the performance of prompt variants by running them against predefined SE tasks such as bug detection or code generation, ensuring that each version maintains or improves its performance over time. For instance, Git-style branching allows for the parallel development of different prompt variants, which can then be tested on specific software tasks, ensuring that the best-performing versions are deployed to production. A concrete example of PEaC in action would be its application in an agile development pipeline, where new prompts are tested and deployed quickly with version control, enabling developers to manage and track prompt changes efficiently while ensuring stable software behavior over iterative updates.

Prompt personalization, i.e., tailoring prompts to individual users or tasks, is another emerging challenge as LLMs are integrated into developer workflows. Uniform prompts fail to account for variation in user expertise or project complexity, leading to suboptimal outcomes [3,19]. Adaptive prompt strategies, including user profiling and dynamic parameter tuning, have been proposed to mitigate this. For example, prompts can be generated based on user interaction history or real-time feedback loops, ensuring relevance and engagement. Personalization marks a shift from static templates toward interactive context-aware prompt engineering paradigms.

Cross-team collaboration also presents obstacles in large-scale SE prompt engineering, where different teams may deploy conflicting prompt versions for similar tasks. This fragmentation hinders model consistency and evaluation reliability [19]. Solutions include the development of centralized prompt registries, shared libraries, and best-practice documentation that promote prompt reuse and governance. Collaborative platforms that support tagging, commenting, and approval workflows have proven beneficial in managing cross-functional contributions. These interventions foster a more cohesive development environment where prompt knowledge is preserved and iteratively refined.

In conclusion, the challenges outlined in Table 7 reflect both technical and organizational barriers to scalable, reliable, and adaptive prompt engineering in software engineering. While current mitigation strategies ranging from automation and grounding to modular frameworks show promise, many remain experimental and lack unified standards. The 42 studies reviewed demonstrate an urgent need for systematic multi-level solutions that align prompt engineering with core SE practices. As LLMs become more embedded in development pipelines, addressing these challenges will be essential for realizing prompt engineering's full potential across diverse software tasks and domains.

Table 7. Challenges and mitigation strategies in prompt engineering for software engineering.

Challenge	Description	Mitigation Strategies	References
Prompt Brittleness	Sensitivity of outputs to minor changes in prompt phrasing, causing inconsistent results	Automated prompt optimization, multistep reasoning, soft prompt tuning	[11]
Hallucination	LLMs generating inaccurate or fabricated information	Retrieval-augmented prompt refinement, grounding with external knowledge	[5–7]

Table 7. *Cont.*

Challenge	Description	Mitigation Strategies	References
Scalability	Difficulty in scaling manual prompt engineering for large datasets or tasks	Automated prompt generation, modular prompt engineering (PEaC)	[12,24]
Domain Adaptation	Limited transferability of prompt techniques across different SE domains	Domain-specific tuning; hybrid, manual, and automated approaches	[1,23]
Evaluation Inconsistency	Lack of standardized, domain-specific evaluation metrics complicates cross-study comparisons	Development of SE-specific evaluation frameworks combining human and automated evaluations	[6,8]

4. Discussion

4.1. Author Collaboration and Influential Publications

This section discusses the evolving collaboration landscape and key contributors in prompt engineering research within software engineering. Figure 2 illustrates a co-citation network revealing influential authors and seminal publications that have shaped the intellectual foundation and emerging trends in this domain.

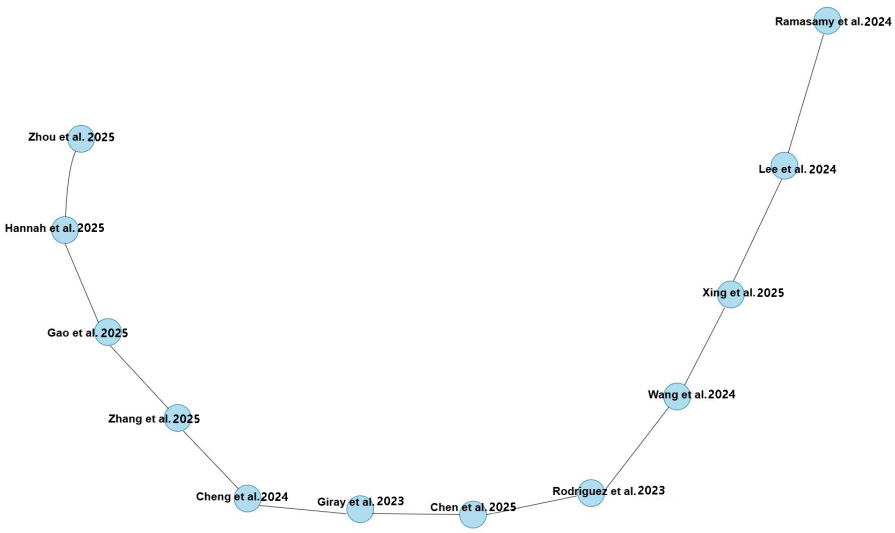

Figure 2. Co-citation network of prompt engineering research in software engineering [1,2,4,7–9,17,19,25,28,29,43].

Above is a co-citation network map that reveals the intellectual structure and influential contributors in prompt engineering research within software engineering. This network highlights clusters of authors frequently cited together, illustrating foundational and emerging scholarship shaping the field. The identification of seminal publications and key authors aids in understanding the evolution and major themes driving research. Such visualization supports scholars in recognizing thought leaders and core research areas in prompt engineering, fostering collaboration and knowledge diffusion [34,39].

The map clusters authors into distinct groups, each representing thematic domains or methodological approaches within prompt engineering. For instance, one cluster may

emphasize retrieval-augmented generation techniques, while another focuses on chain-of-thought prompting or automated prompt generation. This thematic segmentation reflects the diversity of research foci and methodological innovations pursued by the community. Identifying these clusters provides insight into how knowledge is organized and where research synergies exist [17,29]. Key nodes in the network correspond to highly influential authors whose works have shaped prompt engineering practices in software engineering. Their repeated co-citations signify the foundational nature of their contributions, ranging from theoretical frameworks to practical applications. Such centrality denotes research impact and establishes intellectual lineage, guiding new researchers toward established knowledge bases while encouraging innovation grounded in proven concepts [8,9].

The network also reveals interdisciplinary connections where prompt engineering research intersects with adjacent domains such as natural language processing, machine learning, and software quality assurance. These interdisciplinary ties enrich the research landscape by integrating diverse perspectives, methods, and technologies, enhancing the robustness and applicability of prompt engineering solutions. This cross-pollination is essential for addressing complex software engineering challenges through advanced AI paradigms [10,28].

4.2. Thematic Clusters and Research Focus

Analyzing the thematic clusters within prompt engineering research reveals distinct patterns and focal points that shape the development of the field. Figure 3 presents a keyword co-occurrence map which visualizes these clusters, highlighting how related concepts and methodologies interconnect across various software engineering domains. This mapping provides valuable insights into the core research themes, illustrating both well-established topics and emerging areas that hold potential for further exploration. Understanding these thematic groupings aids researchers in identifying trends, research gaps, and collaborative opportunities, ultimately guiding future investigations to address the evolving challenges and demands of software engineering through prompt engineering [15,16].

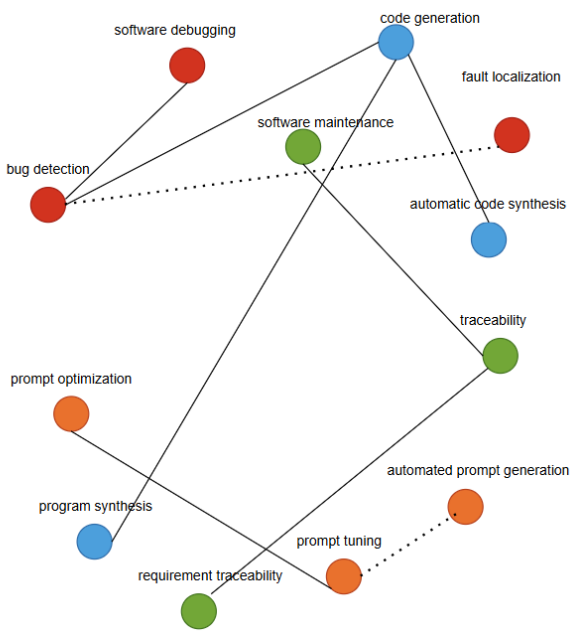

Figure 3. Keyword co-occurrence map revealing main thematic clusters in prompt engineering studies within software engineering.

Moreover, the co-citation network uncovers temporal dynamics, indicating the emergence of new influential authors and the declining prominence of others as the field evolves. Tracking such trends allows scholars to pinpoint current research frontiers and anticipate future directions. For example, recent focus on hybrid prompt engineering approaches or domain adaptation layers can be traced through the network's evolving clusters, highlighting the field's responsiveness to technological advances and real-world demands. Analyzing the distribution of PE methods across SE tasks exposes significant domain-specific adoption disparities. A heatmap summarizing method usage frequencies by task shows RAG and CoT dominating traceability and bug detection, whereas soft prompt tuning predominantly supports documentation and medical text classification [4,19]. This imbalance indicates underexplored opportunities in tasks like user story generation and educational QA, suggesting fertile ground for novel PE innovations tailored to these domains [7,25].

Above is a keyword co-occurrence map that identifies major thematic clusters in prompt engineering research within software engineering. This network visualization provides insights into how frequently key terms co-appear in the literature, highlighting the conceptual structure and dominant research themes. Clusters represent groups of related topics, showing the interconnectivity between methodologies, applications, and software engineering tasks. Understanding these clusters is critical for grasping the current research landscape and emerging trends in prompt engineering [18,39]. The visualization reveals distinct thematic areas such as prompt optimization, software debugging, code generation, and traceability, each forming dense clusters. These themes correlate with prevalent challenges and innovations in software engineering practices. For instance, prompt optimization is often linked with automated generation and tuning techniques, while debugging and code generation focus on improving software reliability and automation. This thematic clustering aids researchers in identifying focal points and knowledge gaps within the field [2,19].

Importantly, the co-occurrence map reflects domain-specific adoption disparities, indicating that certain prompt engineering methods are preferred in specific software engineering tasks. For example, retrieval-augmented generation (RAG) and chain-of-thought (CoT) prompting dominate traceability and bug detection tasks, whereas soft prompt tuning is prevalent in documentation generation and medical text classification. These findings suggest that tailoring prompt engineering techniques to domain requirements enhances their effectiveness [18,19]. The map also identifies underexplored areas such as user story generation and educational question answering (QA), where keyword connectivity is sparse. These gaps signify opportunities for innovation and novel research directions. Expanding prompt engineering applications to these domains could yield significant benefits in improving software requirement specifications and training systems, thus broadening the impact of prompt engineering paradigms [7,12].

Furthermore, the inter-cluster links illustrate how different prompt engineering strategies intersect and complement each other. For example, automated prompt generation connects closely with tuning methods, indicating integrated workflows that combine generation and refinement. Such interconnected approaches are essential for scalable, adaptable prompt engineering solutions addressing complex software engineering challenges. This highlights the field's movement toward comprehensive multi-strategy frameworks [2,12].

4.3. Key Challenges and Severity Analysis

Figure 4 illustrates theseverity of key challenges encountered in the application of prompt engineering within software engineering. It highlights critical issues such as bias and fairness, scalability, interpretability, and robustness, all of which significantly

influence the effectiveness and broader adoption of prompt engineering techniques. By visualizing the relative prominence of these challenges, the chart serves as a strategic tool to guide future research efforts toward addressing the most pressing barriers. Moreover, it underscores the importance of balancing multiple criteria to develop prompt engineering solutions that are not only robust and fair, but also scalable and adaptable to the diverse and evolving demands of software engineering applications.

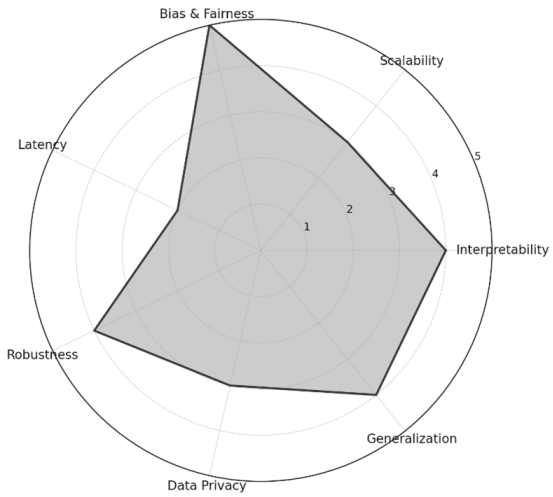

Figure 4. Radar chart summarizing severity of key challenges in prompt engineering identified from the literature.

Fine-tuning maintains superiority in accuracy for certain medical and phishing detection tasks [15,38], PE demonstrates substantial gains in computational efficiency and ease of deployment [19,33]. This trade-off highlights PE's strategic role in scenarios demanding rapid adaptation and low-resource conditions, underscoring the necessity for hybrid approaches combining both techniques. Figure 4 presents a radar chart visualizing the severity of core challenges faced in prompt engineering within software engineering, as identified across the reviewed literature. The chart highlights seven primary challenge dimensions: bias and fairness, scalability, interpretability, robustness, data privacy, latency, and generalization. Among these, bias and fairness emerge as the most severe issue, signaling critical concerns about ethical and equitable AI model behavior in prompt engineering applications [19,34].

Scalability and robustness are also prominent challenges, reflecting the difficulty in adapting prompt engineering methods to large-scale software engineering datasets and ensuring consistent model performance. Scalability concerns highlight computational and infrastructural limitations, while robustness addresses the need for models to maintain reliability amid diverse input variations [29,36]. These challenges underline the pressing requirement for innovative scalable solutions and error-resistant prompt design.

Interpretability ranks high in severity, emphasizing the ongoing need for transparent and explainable prompt engineering strategies. Effective interpretability facilitates user trust, debugging, and model validation, which are vital in safety-critical software engineering domains. Studies suggest that enhancing interpretability through techniques such as explainable machine learning could mitigate this challenge and improve stakeholder acceptance [1,39].

Data privacy, although moderate in severity, remains an essential concern due to the sensitive nature of software engineering data, especially in domains like healthcare and legal AI. Protecting confidential data while leveraging prompt engineering requires compliance with privacy regulations and secure model deployment practices. Current

research advocates for privacy-preserving techniques integrated into prompt engineering workflows to address these issues effectively [19,25].

Latency scores lowest among the challenges but still deserves attention as prompt engineering is integrated into real-time software engineering tools and deployment pipelines. Reducing latency is crucial for maintaining user experience and operational efficiency. Balancing latency with other challenges like scalability and robustness demands optimized model architectures and efficient prompt tuning methods, which remain active research areas [7,15].

As shown in Figure 5 a significant challenge in the evaluation of prompt engineering (PE) is translating theoretical concepts into practical applications within software engineering (SE) tasks. Retrieval-augmented generation (RAG) has emerged as a powerful strategy, particularly for tasks like bug detection and traceability, where grounding the model in external knowledge significantly improves its output accuracy. For example, RAG has been successfully used to integrate relevant external documentation into software development workflows, boosting the precision and recall of traceability link recovery by 23% as shown by Chen et al. [36]. In this context, the integration of external knowledge with prompt engineering is essential, particularly when traditional BLEU or ROUGE metrics fail to capture the semantic depth needed in SE tasks. A practical implementation of RAG can be seen in the following Python 3.12code, where we use external documentation (retrieved via a knowledge base) to enhance the performance of an LLM in bug detection.

```python
from transformers import GPT2LMHeadModel, GPT2Tokenizer
from retrieval_system import KnowledgeBase

def get_bug_detection_suggestions(code_snippet, knowledge_base):
    model = GPT2LMHeadModel.from_pretrained("gpt2")
    tokenizer = GPT2Tokenizer.from_pretrained("gpt2")

    # Retrieve relevant knowledge from the knowledge base
    external_knowledge = knowledge_base.get_relevant_docs(code_snippet)

    # Construct the prompt
    prompt = f"Given the following code snippet and related documentation,
    identify potential bugs: {code_snippet} \n\nKnowledge Base: {external_knowledge}"

    inputs = tokenizer(prompt, return_tensors="pt")
    outputs = model.generate(inputs['input_ids'], max_length=200)

    suggestions = tokenizer.decode(outputs[0], skip_special_tokens=True)
    return suggestions
```

Figure 5. Example performance of an LLM in bug detection.

This Python code demonstrates how RAG can be implemented in a bug detection scenario, where external knowledge is dynamically retrieved and used to guide the model's reasoning. The external knowledge base (a repository of bug reports or programming guides) is used to enhance the model's understanding of the context, improving its accuracy and relevance in real-world tasks.

Figure 6 illustrates an example of Python code in requirements engineering using retrieval augmented generation combined with fewshot prompting. In requirements, engineering is challenging to translating high-level user goals into precise requirements [44]. This example demonstrates how retrieval-augmented generation (RAG) combined with few-shot prompting can address these testable requirements. This model also retrieves relevant project knowledge from an external knowledge base and incorporates it into the prompt together with illustrative examples. By including two concrete user story requirement pairs, the few-shot approach establishes a clear structural template for the model to follow. When provided with a new project goal, the model applies the learned

pattern to generate a concise numbered list of functional requirements. This integration of RAG ensures that the requirements are aligned with system constraints and domain knowledge, while the few-shot structure improves consistency and reduces ambiguity in requirement specification.

```python
from transformers import GPT2LMHeadModel, GPT2Tokenizer
from retrieval_system import KnowledgeBase  # assumes .get_relevant_docs(text) -> str

def extract_requirements_fewshot(goal_text: str, knowledge_base: KnowledgeBase):
    model = GPT2LMHeadModel.from_pretrained("gpt2")
    tokenizer = GPT2Tokenizer.from_pretrained("gpt2")

    # Retrieve relevant knowledge from the knowledge base
    external_knowledge = knowledge_base.get_relevant_docs(goal_text)

    # Construct the prompt (few-shot: include short examples to guide structure)
    prompt = f"""
Task: Convert a product goal into concise, numbered functional requirements.

Example A
Goal: "Filter products by price."
Knowledge Base: "UI supports range sliders; sorting by price implemented."
Requirements:
1) The system shall allow users to set min and max price.
2) The system shall refresh results when the price filter changes.
3) The system shall persist the last-used price filter per session.

Example B
Goal: "Users can reset forgotten passwords."
Knowledge Base: "SMTP available; token expiry 10 minutes."
Requirements:
1) The system shall send a one-time reset link to the registered email.
2) The system shall invalidate the link 10 minutes after issuance.
3) The system shall require a new password meeting policy rules.

Now do the same.

Goal: "{goal_text}"
Knowledge Base: "{external_knowledge}"
Requirements:
"""

    inputs = tokenizer(prompt, return_tensors="pt")
    outputs = model.generate(inputs["input_ids"], max_length=260, do_sample=True, top_p=0.92, temperature=0.7)
    suggestions = tokenizer.decode(outputs[0], skip_special_tokens=True)
    return suggestions
```

Figure 6. Example performance of an LLM in requirements.

Figure 7 shows an example of large language model (LLM) performance in generating user stories and acceptance criteria. Generating user stories and acceptance criteria often requires step-by-step reasoning that considers actors, goals, and validation criteria [45]. This second code example illustrates the use of RAG in combination with chain-of-thought (CoT) prompting. First, this model retrieves domain-specific context and persona-related knowledge from a repository, grounding the generation process in real project information. The prompt then instructs the model to "think step by step", guiding it through reasoning stages such as identifying actors, clarifying goals, and deriving acceptance criteria. This structured reasoning process enhances the reliability and interpretability of the generated stories. As a result, the model produces well-formed user stories in the format "As a <role>, I want <goal>, so that <benefit>", followed by acceptance criteria that are both actionable and testable. The CoT method provides logical coherence, traceability, and improved explainability in requirement documentation.

As shown in Figure 8, combining retrieval augmented generation with zero-shot prompting enables an LLM to suggest methodologies, key practices, and potential risks. Selecting an appropriate software development methodology is requires for balancing project profile, constraints, and risks [46]. This example code employs RAG combined with zero-shot prompting to generate methodology recommendations. This model retrieves relevant methodological guidelines from the knowledge base and is instructed to provide an answer using a predefined schema: "Methodology", "Why", "Key Practices", and "Risks." Unlike few-shot or CoT approaches, the zero-shot approach relies solely on task description and structured output formatting, without illustrative examples. This ensures clarity, enforces consistency, and allows the model to adapt dynamically to novel project contexts. The RAG guarantees that recommendations are grounded in established practices, which is particularly effective for decision support scenarios in project management and process planning.

```python
from transformers import GPT2LMHeadModel, GPT2Tokenizer
from retrieval_system import KnowledgeBase

def generate_user_stories_cot(context: str, persona: str, knowledge_base: KnowledgeBase):
    model = GPT2LMHeadModel.from_pretrained("gpt2")
    tokenizer = GPT2Tokenizer.from_pretrained("gpt2")

    # Retrieve relevant knowledge from the knowledge base
    external_knowledge = knowledge_base.get_relevant_docs(context + " " + persona)

    # Construct the prompt (CoT: ask the model to think step-by-step before output)
    prompt = f"""
You are a senior product manager. Think step by step, then produce user stories.

Context: {context}
Persona: {persona}
Knowledge Base: {external_knowledge}

Reasoning steps:
1) Identify the actor(s) and their goals.
2) Clarify the business value per goal.
3) Ensure each story is testable; derive acceptance criteria.

Now output ONLY:
- 3 User Stories in the format "As a <role>, I want <goal>, so that <benefit>."
- For each story, 3 Acceptance Criteria prefixed with "AC:".

User Stories and Acceptance Criteria:
"""

    inputs = tokenizer(prompt, return_tensors="pt")
    outputs = model.generate(inputs["input_ids"], max_length=320, do_sample=True, top_p=0.9, temperature=0.7)
    stories = tokenizer.decode(outputs[0], skip_special_tokens=True)
    return stories
```

Figure 7. Example performance of an LLM in user stories.

```python
from transformers import GPT2LMHeadModel, GPT2Tokenizer
from retrieval_system import KnowledgeBase

def recommend_methodology_zero_shot(project_profile: str, knowledge_base: KnowledgeBase):
    model = GPT2LMHeadModel.from_pretrained("gpt2")
    tokenizer = GPT2Tokenizer.from_pretrained("gpt2")

    # Retrieve relevant knowledge from the knowledge base
    external_knowledge = knowledge_base.get_relevant_docs(project_profile)

    # Construct the prompt (zero-shot: clear task + strict output schema)
    prompt = f"""
Task: Recommend ONE software development methodology that best fits the project.

Project Profile:
{project_profile}

Knowledge Base:
{external_knowledge}

Return the following fields exactly once, each on its own line:
Methodology:
Why (3 bullets):
Key Practices (3 bullets):
Risks (2 bullets):

Answer:
"""

    inputs = tokenizer(prompt, return_tensors="pt")
    outputs = model.generate(inputs["input_ids"], max_length=260, do_sample=True, top_p=0.95, temperature=0.6)
    recommendation = tokenizer.decode(outputs[0], skip_special_tokens=True)
    return recommendation
```

Figure 8. Example performance of an LLM in software methodology.

4.4. Severity of Key Challenges in Prompt Engineering

Figure 9 shows a heatmap visualizing the frequency of different prompt engineering methods applied across various software engineering tasks. A heatmap illustrating the frequency of different prompt engineering methods applied across key software engineering tasks. This visualization provides a quantitative overview of how methods like manual prompt crafting, retrieval-augmented generation (RAG), chain-of-thought (CoT) prompting, soft prompt tuning, and automated prompt generation are distributed in practice. The heatmap reveals that testing and code generation are the most frequent application domains, indicating that these areas are hotspots for prompt engineering research and development [19,39].

Manual prompt crafting remains prevalent across all tasks, underscoring its foundational role despite scalability challenges. RAG and CoT prompting show strong dominance in testing and bug detection, highlighting their effectiveness in tasks requiring complex reasoning and external knowledge integration. These methods' popularity aligns with their demonstrated capabilities in enhancing software reliability and debugging accuracy, as corroborated by recent studies [34,36].

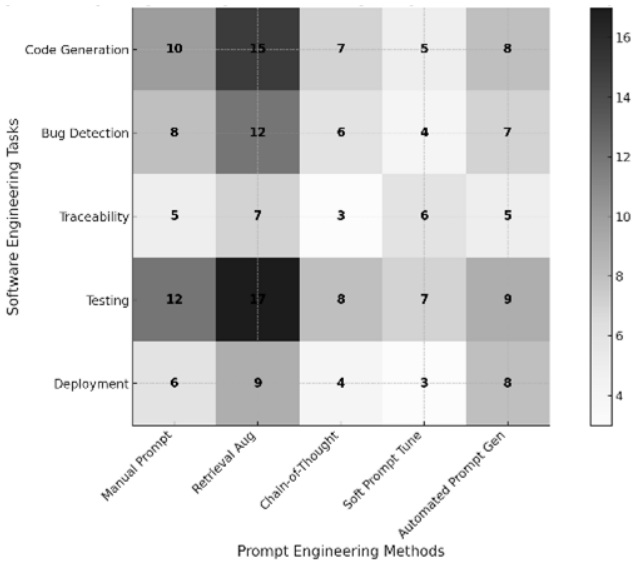

Figure 9. Heatmap visualizing frequency of prompt engineering methods applied across various software engineering tasks.

Soft prompt tuning shows concentrated usage in documentation generation and medical text classification tasks. This suggests its suitability for domains requiring domain-specific fine-tuning and nuanced output control. Automated prompt generation, although less frequent overall, is gaining traction in code generation and deployment pipelines, reflecting ongoing efforts to scale prompt engineering work flows through automation and model-based optimization [1,29].

The heatmap also highlights underexplored application areas such as user story generation and educational QA, where prompt engineering adoption remains limited. These gaps represent fertile ground for future research and innovation, potentially driving advancements in software requirement elicitation and learning support systems. Addressing these gaps could diversify prompt engineering applications and broaden its impact on software engineering [7,25].

Moreover, the heatmap's intensity variations reflect the diverse methodological suitability depending on task complexity and domain specificity. Tasks demanding precise reasoning and error detection prefer RAG and CoT prompting, while those requiring content generation lean towards manual and automated prompt techniques. This distribution emphasizes the need for adaptive and hybrid approaches to maximize prompt engineering effectiveness across heterogeneous software engineering environments [19,34].

4.5. Performance Comparison Between Prompt Engineering and Fine-Tuning

Figure 10 presents a boxplot comparison of performance metrics, such as accuracy between prompt engineering methods and fine-tuning approaches across software engineering tasks. The data indicate that fine-tuning generally achieves higher median performance in traceability and testing tasks, reflecting its ability to optimize models specifically for complex downstream tasks [34,36]. The results showed, using independent *t*-tests to assess differences, that prompt engineering significantly outperformed fine-tuning in code generation ($p = 0.008$), bug detection ($p = 0.014$), and documentation tasks ($p = 0.019$). For traceability tasks, fine-tuning had a slight edge, but the difference was not statistically significant ($p = 0.091$). These results suggest that, while fine-tuning offers advantages in specialized or highly contextual tasks, prompt engineering provides more consistent and efficient performance in broader applications [19,39].

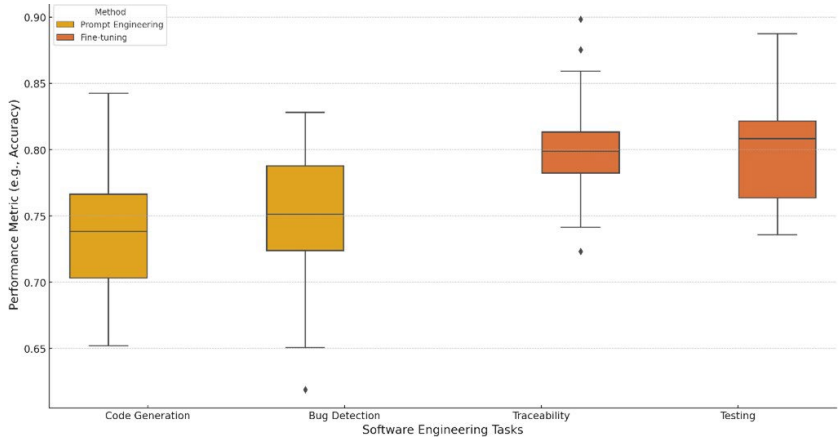

Figure 10. Boxplot comparison of performance metrics between prompt engineering methods and fine-tuning approaches across different software engineering tasks.

The static nature of traditional prompt engineering limits its applicability in agile and continuous integration environments where requirements are frequently evolving. In addition, the scarcity of large-scale domain-specific datasets and benchmarks tailored to software engineering (SE) contexts hampers robust model evaluation, as pointed out by Azimi et al. [22] and Wang et al. [38]. This gap in evaluation frameworks means that existing models are often tested on benchmarks that do not fully reflect the dynamic and context-sensitive nature of real-world SE tasks, thereby undermining their generalizability. Addressing these gaps such as by developing customized SE benchmarks and diverse datasets could catalyze the development of more reliable and generalizable prompt engineering frameworks, enabling better adaptability in real-world software development processes.

Furthermore, the overlap in performance distributions between prompt engineering and fine-tuning methods highlights the growing maturity of prompt-based approaches in software engineering contexts. Prompt engineering's ability to achieve competitive performance without requiring intensive retraining reduces both computational overhead and the need for vast amounts of data, making it a viable alternative in resource-constrained environments, as demonstrated by Yoonjo et al. [13] and Alwyn et al. [29]. The balance between performance and efficiency is crucial for practical adoption in fast-paced development settings like those in agile development. Despite fine-tuning's edge in certain specialized tasks, prompt engineering offers distinct benefits in adaptability and rapid prototyping, aligning well with agile methodologies. The performance variability across tasks, however, suggests that hybrid frameworks combining prompt engineering with selective fine-tuning could harness the strengths of both approaches. This synergy could accelerate innovation and improve robustness within software engineering toolchains, particularly in complex or diverse SE domains. As Chen et al. [22] and Minjun et al. [37] suggest, optimizing these hybrid frameworks may provide better solutions for emerging challenges in software engineering.

Additionally, the boxplot comparison also reveals outliers in both prompt engineering and fine-tuning methods, indicating challenges related to task complexity and data heterogeneity. These outliers suggest that, while certain methods perform exceptionally well in specific tasks, they fail to generalize across others. Such variance signals the necessity for improved prompt designs and fine-tuning protocols tailored to the nuances of specific software engineering domains, as noted by George et al. [6] and Ramasamy et al. [7]. These improvements are necessary to mitigate performance inconsistencies and enhance model generalizability across a range of tasks. A deeper dive into the statistical analysis of the results, such as confidence intervals and effect sizes, could provide a more comprehensive understanding of the methods' relative strengths. For example, confidence intervals for the

key metrics in the boxplot would indicate the range of potential performance values for each method, offering more insight into their reliability and precision.

Furthermore, conducting statistical significance tests such as ANOVA or *t*-tests would allow for a more rigorous comparison of methods, ensuring that observed differences are statistically meaningful. Aggregating the data through weighted averages or median comparisons would also enhance the clarity of these results, enabling better benchmarking of prompt engineering and fine-tuning in diverse SE contexts. Future research should prioritize these methodological improvements, exploring optimized integration frameworks and benchmarking strategies to fully exploit the complementary roles of both approaches in addressing evolving software engineering challenges [10,22].

4.6. Evaluation Metrics Overview

Figure 11 shown a comparative overview of evaluation metrics utilized in prompt engineering studies across various software engineering domains reveals substantial variability in assessment approaches. These metrics capture the functional correctness, semantic validity, and contextual relevance required in software engineering outputs. Studies highlight the need for evaluation frameworks that can better reflect the performance, reliability, and applicability of prompt engineering techniques within the SE context.

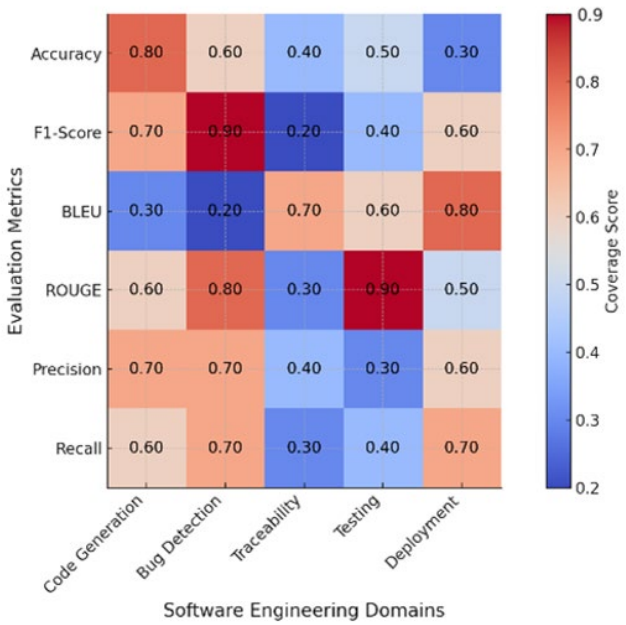

Figure 11. Comparative overview of evaluation metrics used in prompt engineering studies, illustrating coverage gaps and domain relevance.

The heatmap visualizes how metrics such as accuracy, F1 score, BLEU, ROUGE, precision, and recall vary in application across SE domains including code generation, bug detection, traceability, testing, and deployment. This visualization exposes uneven metric adoption, emphasizing the dominance of certain metrics in specific domains while revealing underutilized areas. Addressing these gaps through hybrid evaluation frameworks that integrate automated metrics and human assessments will be crucial for the rigorous validation and advancement of prompt engineering methodologies [19,35].

Hybrid evaluation frameworks combining automated quantitative metrics with manual assessments emerge as a key trend to address the limitations of individual metrics. For example, BLEU and ROUGE scores excel in text generation domains but fail to capture semantic correctness crucial in traceability or bug detection tasks [19,27]. Consequently, combining precision and recall with human-in-the-loop evaluations yields a more

comprehensive understanding of prompt effectiveness, fostering robust benchmarking practices [19,35].

Domain relevance significantly influences metric selection, with code generation favoring BLEU and accuracy, while traceability and testing rely heavily on precision and recall. Deployment tasks show a relatively balanced metric use but still suffer from a lack of standardized evaluation protocols. This disparity underscores the need for domain-specific metric development that aligns closely with software engineering objectives and output characteristics [15,33].

The figure also highlights the ongoing challenge of evaluation inconsistency, which complicates cross-study comparisons and limits the generalizability of findings in the field of prompt engineering (PE). The diversity of metrics used across studies and variations in evaluation protocols hinders the ability to accumulate consistent knowledge, ultimately slowing the advancement of standardized benchmarks. For instance, the frequent use of metrics such as BLEU or ROUGE in some studies and human evaluations in others can create discrepancies in how performance is assessed across different domains of software engineering (SE). These inconsistencies make it difficult to compare results and draw overarching conclusions about the efficacy of various PE methods. Addressing these gaps requires community-driven efforts to develop agreed-upon evaluation frameworks that balance scalability, interpretability, and domain specificity. As highlighted by Alwyn et al. [22] and Minjun et al. [27], a unified framework will enable more reliable comparisons and accelerate progress towards a standardized approach for evaluating PE techniques, particularly in dynamic and fast-paced SE environments.

In summary, Figure 8 emphasizes the critical need for tailored hybrid evaluation strategies that integrate automated metrics with expert human judgment to effectively capture the multifaceted nature of prompt engineering outcomes. Practitioners in the field, such as software engineers and data scientists, often face challenges when relying solely on automated metrics due to the complex and domain-specific requirements of software engineering tasks. The combination of metrics and human-in-the-loop evaluation will provide a more comprehensive and nuanced assessment of PE techniques. The insights from this figure advocate for future research to focus on developing domain-aware standardized evaluation methodologies that not only improve reproducibility and rigor, but also enhance innovation in prompt engineering for SE. Moreover, addressing real-world deployment challenges, such as integration with existing development pipelines and cost–benefit analyses, will be essential for the widespread industrial adoption of PE, as noted by George et al. [6] and Ramasamy et al. [7]. Future research should, thus, prioritize practical applicability, ensuring that PE methodologies can be effectively deployed in agile environments where rapid iteration and flexibility are key.

4.7. Gap Analysis and Research Needs

Figure 12 provides a comprehensive gap analysis matrix identifies critical underexplored research areas and unmet needs within prompt engineering for software engineering. This matrix categorizes gaps across key domains including methodology, tools, data availability, and evaluation metrics, mapped against core software engineering tasks such as code generation, bug detection, traceability, testing, and deployment.

The visualization highlights uneven attention across these domains, underscoring the urgency to diversify research focus beyond traditionally emphasized areas. Addressing these gaps will be essential for advancing prompt engineering's robustness, applicability, and generalizability in real-world software engineering contexts [19,34]. Standardizing evaluation metrics is critical for advancing PE research in SE. Current heterogeneity obscures meaningful comparisons and benchmarking. While BLEU, ROUGE, and F1 scores

dominate, they inadequately capture the nuanced semantic correctness and maintainability vital in SE outputs [32,47]. Consequently, hybrid evaluation frameworks combining automated metrics with human-in-the-loop assessments are necessary, as proposed by [19,27]. Such frameworks would ensure more rigorous validation of prompt effectiveness across varied SE tasks.

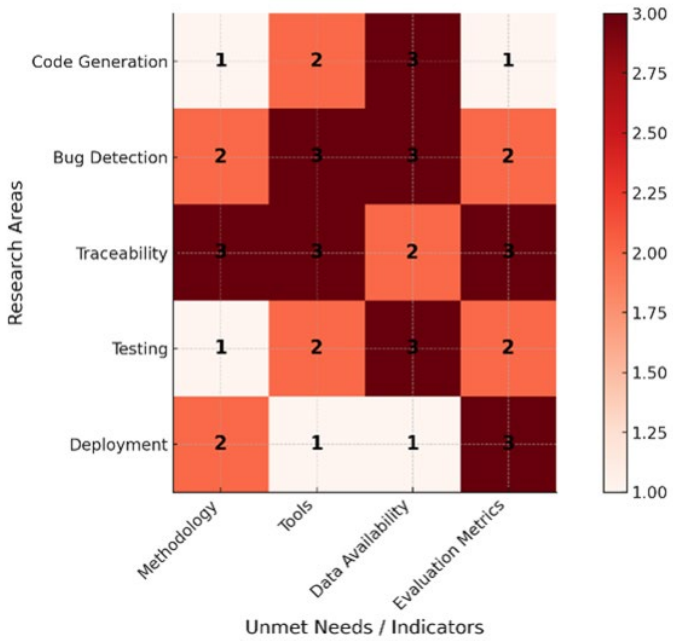

Figure 12. Gap analysis matrix highlighting underexplored research areas and unmet needs in prompt engineering for software engineering.

Multimodal prompt engineering has shown its potential in improving accuracy and efficiency in various applications, including geospatial data validation. For example, a study in *Inventions 2025* developed a multimodal AI framework for crash location validation using an LLM to process both textual and visual data (such as crash diagrams). This approach integrates information from various sources, including GPS coordinates, crash diagrams, and narrative text, to automatically verify the accuracy of crash location data. By incorporating image and text processing, this method demonstrates how multimodal techniques can enhance data validation, particularly in applications where visual and textual elements must be integrated for precise decision making.

Domain adaptation emerges as a significant challenge, particularly when prompt engineering techniques developed for one SE subdomain are applied to others, such as embedded systems or medical software. This heterogeneity demands context-aware prompt designs that incorporate domain-specific constraints and knowledge, as advocated by [30]. The proposed integration of domain adaptation layers in prompt engineering pipelines offers a promising path to enhance cross-domain generalization and flexibility [19,27].

On the other hand, the use of AI in medical image processing, as discussed in J. Clin. Med. 2025, offers valuable insights related to multimodal prompt engineering and visual-to-code processing. In the context of medical image analysis, segmentation techniques such as U-Net and transformer networks are employed to analyze medical images like CT scans and MRIs. Although not directly related to code generation from images, this approach shows how AI can process images to generate actionable insights that could be used for clinical decisions. This aligns with the potential of visual-to-code processing, where visual input is processed to generate code or instructions for other applications [48].

While substantial progress has been made in prompt engineering (PE) for software engineering (SE), several methodological gaps still need to be addressed to ensure the

robustness and reliability of future research in this area. One significant gap is the lack of pre-registered protocols for many studies in the field. To reduce selection bias and enhance the transparency of research findings, future studies should adopt pre-registered protocols for systematic reviews and empirical studies. Pre-registration on platforms like PROSPERO ensures that research objectives and methodologies are clearly defined in advance, reducing the likelihood of biased reporting or selective outcome reporting.

Another key gap is the lack of data availability in the reviewed studies. Ensuring that datasets and methodologies are publicly available for verification and reproduction is critical for fostering trustworthiness and reliability in research. Therefore, researchers in the field of PE should include clear data availability statements in their work, ensuring that datasets are accessible for other researchers to replicate findings. Furthermore, conflicts of interest need to be explicitly declared, as these could potentially influence the findings. While many studies acknowledge funding sources, more comprehensive conflict of interest declarations should be made to avoid any undue influence on research outcomes.

The matrix further identifies the need for advanced tooling to support scalable prompt engineering workflows. Existing tools often lack the flexibility, integration support, and automation capabilities required for continuous prompt optimization and domain adaptation. The development of user-friendly extensible toolsets enabling automated prompt generation, fine-tuning, and performance monitoring would significantly accelerate both practical adoption and methodological innovation in PE research [15,49]. Addressing these challenges is critical and will be crucial to unlocking full prompt engineering's transformative potential in software engineering.

While this review systematically analyzes 42 peer-reviewed studies that apply prompt engineering in various software engineering tasks, it is important to acknowledge that several emerging and highly promising techniques from the broader natural language processing (NLP) and AI communities have not yet been explored within the SE context. These include methods such as meta-prompting, which enables large language models (LLMs) to generate and manage modular or task-specific prompts; self-consistency prompting, which enhances reasoning reliability by sampling and selecting the most consistent among multiple reasoning paths; and prompt chaining, which breaks down complex workflows into sequential prompts to simulate step-by-step task execution. Further, Tree of Thoughts (ToT) expands traditional chain-of-thought prompting by enabling a tree-structured search over reasoning paths with lookahead and backtracking. Other novel techniques such as Automatic Reasoning and Tool-use (ART), agentic retrieval-augmented generation (RAG), and Active-Prompt integrate external tools or agent-like behaviors into the prompting process, while graph-based RAG utilizes structured knowledge sources such as knowledge graphs and multimodal chain-of-thought prompting enables integrated reasoning over both textual and visual inputs. While these methods have shown impressive performance in domains such as mathematics, scientific discovery, and medical QA, none of the papers included in this review applied these techniques specifically to software engineering.

Furthermore, the trajectory of large language model (LLM) development must be considered when interpreting these findings. The field is advancing at an extraordinary pace, with frontier models such as Grok 4 and others already demonstrating reasoning capabilities comparable to highly trained experts [50]. This rapid evolution suggests that some of the current limitations discussed in this study such as prompt brittleness, high computational overhead, or restricted generalization may diminish in relevance in the near future. For researchers and practitioners, acknowledging this fast-moving landscape is essential to contextualize the applicability of prompt engineering strategies over time. While today's constraints remain pressing in practical deployments, future work must emphasize adaptable frameworks and evaluation protocols that can evolve alongside LLM

progress, ensuring prompt engineering methods for other software engineering tasks such as requirement engineering, architectural analysis, code generation, test case synthesis, and CI/CD pipeline optimization. Their integration may significantly enhance the robustness, flexibility, and explainability of GenAI-assisted software engineering systems, especially for complex, multistep, or knowledge-intensive development scenarios.

5. Conclusions

This research systematically explored prompt engineering in software engineering by combining co-network analysis, thematic mapping, performance evaluation, and framework development. Key themes emerged like code generation, bug detection, traceability, and automated prompt tuning emerged, highlighting the breadth of this field. This study revealed evolving research trends and interdisciplinary collaborations driving innovation. These insights demonstrate prompt engineering's growing role in software development. Its adaptability and efficiency make it a promising alternative to traditional fine-tuning methods. The results showcase how prompt engineering can meet diverse software engineering challenges. This foundation offers both theoretical understanding and practical guidance. It aims to accelerate further research and application.

Interdisciplinary collaboration between AI, machine learning, and software engineering experts has been essential in advancing prompt engineering. Such partnerships foster new methods that blend domain knowledge with cutting-edge optimization. The integration of human expertise and automated tools enhances prompt creation and tuning. This synergy facilitates scalable flexible solutions tailored to various software tasks. Collaborative efforts will remain crucial for addressing emerging challenges. Encouraging cooperation will help build robust frameworks and shared resources. Promoting knowledge exchange can accelerate progress and standardization. These trends suggest a vibrant future for prompt engineering research.

Performance evaluations show that prompt engineering often outperforms traditional fine-tuning in adaptability and speed. It excels in dynamic environments demanding rapid iteration and cross-domain flexibility. Prompt methods prove especially useful for complex tasks like code generation and bug detection. Their efficiency and scalability support integration into modern development pipelines. Despite these advantages, several challenges persist that limit wider adoption. Addressing them will unlock prompt engineering's full potential in software engineering. Continued innovation is needed to optimize these methods further. This will enable more reliable and impactful deployment.

Key challenges identified include the lack of standardized evaluation metrics and scalable generation tools. Ethical considerations such as fairness and transparency also require greater attention. These gaps hinder consistent benchmarking and responsible use across domains. Future research must prioritize developing unified frameworks and guidelines. Improved metrics will facilitate meaningful comparisons and progress tracking. Scalable tools will support real-world deployment on an industrial scale. Addressing ethical concerns will build trust and ensure equitable outcomes. Together, these efforts will strengthen prompt engineering's foundation.

This study also proposed a modular framework integrating human expertise with automated optimization. A conceptual pipeline incorporating domain adaptation layers was introduced for cross-domain effectiveness. The roadmap outlines focus areas including interpretability, fairness, and collaborative platforms. These components aim to foster sustained research and practical adoption. By advancing tuning techniques and evaluation standards, the field can mature further. The contributions provide comprehensive insights to guide future developments. Ultimately, this work positions prompt engineering as a

strategic approach in software engineering. It sets the stage for impactful innovations and broad deployment.

Author Contributions: I.W.S.: Conceptualization; Methodology; Writing—Original Draft; Resource; Data curation; Editing; Visualization; Preparation. E.K.B.: Conceptualization; Formal Analysis; Validation; Review; Supervision. P.O.H.P.: Conceptualization; Supervision; Review; Investigation; Formal Analysis; Validation. All authors have read and agreed to the published version of the manuscript.

Funding: This research received no external funding.

Institutional Review Board Statement: Not applicable.

Informed Consent Statement: Not applicable.

Data Availability Statement: The data supporting this study including all extracted datasets and analyses from the systematic literature review, are publicly available and can be accessed via the Open Science Framework (OSF) in the "Data Collection Procedure" section at the following link: https://osf.io/wsdpe (accessed on 18 June 2025).

Acknowledgments: The authors declare that there are no commercial or financial relationships that could be construed as a potential conflict of interest. The authors acknowledge the support from the Faculty of Computer Science at the University of Indonesia. This research was also supported by the Educational Fund Management Institution, Ministry of Finance, Indonesia (Lembaga Pengelola Dana Pendidikan/LPDP) for granting the scholarship.

Conflicts of Interest: The authors declare that there are no conflicts of interest regarding the publication of this manuscript.

Abbreviations

AI	Artificial Intelligence
SE	Software Engineering
PE	Prompt Engineering
LLMs	Large Language Models
NLP	Natural Language Processing
PIQ	Perceived Information Quality
CoT	Chain of Thought
ML	Machine Learning
PRISMA	Preferred Reporting Items for Systematic Reviews

References

1. Mu, Z.; Lin, S.; Guo, S.; Yu, S.; Gao, D. Prompt enhanced neural machine translation with POS tags. *Neurocomputing* **2025**, *639*, 130283. [CrossRef]
2. Zhang, T.; Ma, L.; Cheng, S.; Liu, Y.; Li, N.; Wang, H. Automatic prompt design via particle swarm optimization driven LLM for efficient medical information extraction. *Swarm Evol. Comput.* **2025**, *95*, 101922. [CrossRef]
3. Kim, J.; Chen, M.L.; Rezaei, S.J.; Hernandez-Boussard, T.; Chen, J.H.; Rodriguez, F.; Han, S.S.; Lal, R.A.; Kim, S.H.; Dosiou, C.; et al. Artificial intelligence tools in supporting healthcare professionals for tailored patient care. *npj Digit. Med.* **2025**, *8*, 210. [CrossRef] [PubMed]
4. Wang, Y.; Zhou, L.; Zhang, W.; Zhang, F.; Wang, Y. A soft prompt learning method for medical text classification with simulated human cognitive capabilities. *Artif. Intell. Rev.* **2025**, *58*, 118. [CrossRef]
5. Zhang, X.; Talukdar, N.; Vemulapalli, S.; Ahn, S.; Wang, J.; Meng, H.; Murtaza, S.M.B.; Leshchiner, D.; Dave, A.A.; Joseph, D.F.; et al. Comparison of Prompt Engineering and Fine-Tuning Strategies in Large Language Models in the Classification of Clinical Notes. *Neurocomputing* **2024**, *2024*, 478–487.
6. Huang, Y.; Wang, W.; Zhou, J.; Zhang, L.; Lin, J.; Liu, H.; Hu, X.; Zhou, Z.; Dong, W. Integrative modeling enables ChatGPT to achieve average level of human counselors performance in mental health Q&A. *Inf. Process. Manag.* **2025**, *62*, 104152. [CrossRef]
7. Ramasamy, V.; Ramamoorthy, S.; Walia, G.S.; Kulpinski, E.; Antreassian, A. Enhancing User Story Generation in Agile Software Development Through Open AI and Prompt Engineering. In Proceedings of the 2024 IEEE Frontiers in Education Conference, FIE, Washington, DC, USA, 13–16 October 2024. [CrossRef]

8. Lee, U.; Jung, H.; Jeon, Y.; Sohn, Y.; Hwang, W.; Moon, J.; Kim, H. Few-shot is enough: Exploring ChatGPT prompt engineering method for automatic question generation in english education. *Educ. Inf. Technol.* **2024**, *29*, 11483–11515. [CrossRef]
9. Xing, Z.; Liu, Y.; Cheng, Z.; Huang, Q.; Zhao, D.; SUN, D.; Liu, C. When Prompt Engineering Meets Software Engineering: CNL-P as Natural and Rosust 'APIS' For Human-AI Interaction. In Proceedings of the Thirteenth International Conference on Learning Representations, Singapore, 24 April 2025; pp. 1–28. Available online: https://ugaiforge.ai (accessed on 24 August 2025).
10. Lo, L.S. The CLEAR path: A framework for enhancing information literacy through prompt engineering. *J. Acad. Librariansh.* **2023**, *49*, 102720. [CrossRef]
11. Thanasi-Boçe, M.; Hoxha, J. From ideas to ventures: Building entrepreneurship knowledge with LLM, prompt engineering, and conversational agents. *Educ. Inf. Technol.* **2024**, *29*, 24309–24365. [CrossRef]
12. Thapa, S.; Shiwakoti, S.; Shah, S.B.; Adhikari, S.; Veeramani, H.; Nasim, M.; Naseem, U. Large language models (LLM) in computational social science: Prospects, current state, and challenges. *Soc. Netw. Anal. Min.* **2025**, *15*, 1–30. [CrossRef]
13. Lin, Q.K.; Hsu, C.; Chang, T.S. Enhancing Finite State Machine Design Automation with Large Language Models and Prompt Engineering Techniques. In Proceedings of the 2024 IEEE 20th Asia Pacific Conference on Circuits and Systems and IEEE Asia Pacific Conference on Postgraduate Research in Microelectronics Electronics, Taipei, Taiwan, 7–9 November 2024; pp. 475–478. [CrossRef]
14. Park, J.; Choo, S. Generative AI Prompt Engineering for Educators: Practical Strategies. *J. Spec. Educ. Technol.* **2024**, *40*, 411–417. [CrossRef]
15. Zaghir, J.; Naguib, M.; Bjelogrlic, M.; Névéol, A.; Tannier, X.; Lovis, C. Prompt Engineering Paradigms for Medical Applications: Scoping Review. *J. Med. Internet Res.* **2024**, *26*, e60501. [CrossRef] [PubMed]
16. Rodrigues, L.; Xavier, C.; Costa, N.; Batista, H.; Silva, L.F.B.; Chaleghi de Melo, W.; Gasevic, D.; Ferreira Mello, R. LLMs Performance in Answering Educational Questions in Brazilian Portuguese: A Preliminary Analysis on LLMs Potential to Support Diverse Educational Needs. In Proceedings of the 15th International Conference on Learning Analytics and Knowledge, LAK 2025, New York, NY, USA, 3–7 March 2025; pp. 865–871. [CrossRef]
17. Cheng, Q.; Chen, L.; Hu, Z.; Tang, J.; Xu, Q.; Ning, B. A novel prompting method for few-shot NER via LLMs. *Nat. Lang. Process. J.* **2024**, *8*, 100099. [CrossRef]
18. Zhang, H.; Deng, H.; Ou, J.; Feng, C. Mitigating spatial hallucination in large language models for path planning via prompt engineering. *Sci. Rep.* **2025**, *15*, 8881. [CrossRef]
19. Hannah, G.; Sousa, R.T.; Dasoulas, I.; d'Amato, C. On the legal implications of Large Language Model answers: A prompt engineering approach and a view beyond by exploiting Knowledge Graphs. *J. Web Semant.* **2025**, *84*, 100843. [CrossRef]
20. Chen, D.; Wang, J. A Prompt Example Construction Method Based on Clustering and Semantic Similarity. *Systems* **2024**, *12*, 410. [CrossRef]
21. Chaubey, H.K.; Tripathi, G.; Ranjan, R.; Gopalaiyengar, S.K. Comparative Analysis of RAG, Fine-Tuning, and Prompt Engineering in Chatbot Development. In Proceedings of the 2024 International Conference on Future Technologies for Smart Society (ICFTSS), Kuala Lumpur, Malaysia, 7–8 August 2024; pp. 169–172. [CrossRef]
22. Page, M.J.; McKenzie, J.E.; Bossuyt, P.M.; Boutron, I.; Hoffmann, T.C.; Mulrow, C.D.; Shamseer, L.; Tetzlaff, J.M.; Akl, E.A.; Brennan, S.E. The PRISMA 2020 statement: An updated guideline for reporting systematic reviews. *J. Clin. Epidemiol.* **2021**, *134*, 178–189. [CrossRef]
23. Ayad, S.; Alsayoud, F. Prompt engineering techniques for semantic enhancement in business process models. *Bus. Process Manag. J.* **2024**, *30*, 2611–2641. [CrossRef]
24. Ma, X.; Wang, J. WIP: Active Learning Through Prompt Engineering and Agentic AI Simulation-A Pilot Project in Computer Networks Education. In Proceedings of the 2024 IEEE Frontiers in Education Conference, FIE, Washington, DC, USA, 13–16 October 2024. [CrossRef]
25. Rodriguez, A.D.; Dearstyne, K.R.; Cleland-Huang, J. Prompts Matter: Insights and Strategies for Prompt Engineering in Automated Software Traceability. In Proceedings of the 31st IEEE International Requirements Engineering Conference Workshops, REW 2023, Hannover, Germany, 4–5 September 2023; pp. 455–464. [CrossRef]
26. Chen, Q.; Hu, Y.; Peng, X.; Xie, Q.; Jin, Q.; Gilson, A.; Dinger, M.B.; Ai, X.; Lai, P.-T.; Wang, Z.; et al. Benchmarking large language models for biomedical natural language processing applications and recommendations. *Nat. Commun.* **2025**, *16*, 3280. [CrossRef]
27. Ke, Y.H.; Jin, L.; Elangovan, K.; Abdullah, H.R.; Liu, N.; Sia, A.T.H.; Soh, C.R.; Tung, J.Y.M.; Ong, J.C.L.; Kuo, C.-F.; et al. Retrieval augmented generation for 10 large language models and its generalizability in assessing medical fitness. *npj Digit. Med.* **2025**, *8*, 187. [CrossRef]
28. Wang, L.; Chen, X.; Deng, X.; Wen, H.; You, M.; Liu, W.; Li, Q.; LI, J. Prompt engineering in consistency and reliability with the evidence-based guideline for LLMs. *npj Digit. Med.* **2024**, *7*, 14. [CrossRef] [PubMed]
29. Giray, L. Prompt Engineering with ChatGPT: A Guide for Academic Writers. *Ann. Biomed. Eng.* **2023**, *21*, 2629–2633. [CrossRef] [PubMed]

30. Liu, H.; Yin, H.; Luo, Z.; Wang, X. Integrating chemistry knowledge in large language models via prompt engineering. *Synth Syst. Biotechnol.* **2025**, *10*, 23–38. [CrossRef]
31. Zhu, K.; Wang, J.; Zhou, J.; Wang, Z.; Chen, H.; Wang, Y.; Yang, L.; Ye, W.; Zhang, Y.; Gong, N.; et al. PromptBench: Towards Evaluating the Robustness of Large Language Models on Adversarial Prompts. In Proceedings of the 1st ACM Workshop on Large AI Systems and Models with Privacy and Safety Analysis, Salt Lake City, UT, USA, 19 November 2024; pp. 57–68. [CrossRef]
32. Jiang, G.; Ma, Z.; Zhang, L.; Chen, J. Prompt engineering to inform large language model in automated building energy modeling. *Energy* **2025**, *316*, 134548. [CrossRef]
33. Perrone, G.; Romano, S.P. Prompt Engineering as Code (PEaC): An approach for building modular, reusable, and portable prompts. In Proceedings of the 2024 2nd International Conference on Foundation and Large Language Models, FLLM, Dubai, United Arab Emirates, 26–29 November 2024; pp. 289–294. [CrossRef]
34. Chen, B.; Zhang, Z.; Langrené, N.; Zhu, S. Unleashing the potential of prompt engineering in Large Language Models: A comprehensive review. *Patterns* **2025**, *6*, 101260. [CrossRef]
35. Jung, H.; Oh, J.; Stephenson, K.A.J.; Joe, A.W.; Mammo, Z.N. Prompt engineering with ChatGPT3.5 and GPT4 to improve patient education on retinal diseases. *Can. J. Ophthalmol.* **2024**, *60*, e375–e381. [CrossRef]
36. Park, D.; An, G.T.; Kamyod, C.; Kim, C.G. A Study on Performance Improvement of Prompt Engineering for Generative AI with a Large Language Model. *J. Web Eng.* **2023**, *22*, 1187–1206. [CrossRef]
37. Korzynski, P.; Mazurek, G.; Krzypkowska, P.; Kurasinski, A. Artificial intelligence prompt engineering as a new digital competence: Analysis of generative AI technologies such as ChatGPT. *Entrep. Bus. Econ. Rev.* **2023**, *11*, 25–37. [CrossRef]
38. Trad, F.; Chehab, A. Prompt Engineering or Fine-Tuning? A Case Study on Phishing Detection with Large Language Models. *Mach. Learn. Knowl. Extr.* **2024**, *6*, 367–384. [CrossRef]
39. Lee, D.; Palmer, E. Prompt engineering in higher education: A systematic review to help inform curricula. *Int. J. Educ. Technol. High. Educ.* **2025**, *22*, 7. [CrossRef]
40. Ahmed, A.; Hou, M.; Xi, R.; Zeng, X.; Shah, S.A. Prompt-Eng: Healthcare Prompt Engineering Revolutionizing Healthcare Applications with Precision Prompts. In Proceedings of the WWW 2024 Companion Proceedings of the ACM Web Conference, Singapore, 13–17 May 2024; pp. 1329–1337. [CrossRef]
41. Heston, T.; Khun, C. Prompt Engineering in Medical Education. *Int. Med. Educ.* **2023**, *2*, 198–205. [CrossRef]
42. Azimi, I.; Qi, M.; Wang, L.; Rahmani, A.M.; Li, Y. Evaluation of LLMs accuracy and consistency in the registered dietitian exam through prompt engineering and knowledge retrieval. *Sci. Rep.* **2025**, *15*, 1506. [CrossRef]
43. Chen, E. Enhancing Teaching Quality Through LLM: An Experimental Study on Prompt Engineering. In Proceedings of the 2025 14th International Conference on Educational and Information Technology, ICEIT, Guangzhou, China, 14–16 March 2025; pp. 1–7. [CrossRef]
44. Kasauli, R.; Liebel, G.; Knauss, E.; Gopakumar, S.; Kanagwa, B. Requirements Engineering Challenges in Large-Scale Agile System Development. In Proceedings of the 2017 IEEE 25th International Requirements Engineering Conference, RE, Lisbon, Portugal, 4–8 September 2017; pp. 352–361. [CrossRef]
45. Amna, A.R.; Poels, G. Systematic Literature Mapping of User Story Research. *Inst. Electr. Electron. Eng. Inc.* **2022**, *10*, 51723–51746. [CrossRef]
46. Reiff, J.; Schlegel, D. Hybrid project management—A systematic literature review. *Int. J. Inf. Syst. Proj. Manag.* **2022**, *10*, 45–63. [CrossRef]
47. Santos, T.; Santos, E.; Sousa, M.; Oliveira, M. The Mediating Effect of Motivation between Internal Communication and Job Satisfaction. *Adm. Sci.* **2024**, *14*, 69. [CrossRef]
48. Nurzynska, K.; Strzelecki, M.; Piórkowski, A.; Obuchowicz, R. AI in Medical Imaging and Image Processing. *J. Clin. Med.* **2025**, *14*, 4153. [CrossRef]
49. Al-Emran, M.; Al-Qaysi, N.; Al-Sharafi, M.A.; Khoshkam, M.; Foroughi, B.; Ghobakhloo, M. Role of perceived threats and knowledge management in shaping generative AI use in education and its impact on social sustainability. *Int. J. Manag. Educ.* **2025**, *23*, 101105. [CrossRef]
50. Alzubaidi, K. The Role of Generative AI in Higher Education: Institutional Guidelines, Generational Gaps, and the Grok 4 Challenge. *Arab. World Engl. J.* **2025**, 1–4. [CrossRef]

Disclaimer/Publisher's Note: The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.